

# Formal Specification and Behavioural Analysis of the Inter-Blockchain Communication Protocol: Are Asset Transfer and Data Transport Semantically Unified, Stacked, or Conflicting?

Jazztine Kirvy Gonzales

Bachelor of Science in Computer Science

Jonel C. Ganalon

Bachelor of Science in Computer Science

Senior Thesis submitted to the faculty of the

Department of Computer Science

College of Computer Studies, Ateneo de Naga University

in partial fulfillment of the requirements for their respective

Bachelor of Science degrees

---

Project Advisor: Adviser's Name

First Panel Member

Second Panel Member

Third Panel Member

25 May 2026

Naga City, Philippines

Keywords: Blockchain, Interoperability, Semantics, IBC, Process Algebra

Copyright 2026, Jazztine Kirvy Gonzales and Jonel C. Ganalon

The Senior Project entitled

**Formal Specification and Behavioural Analysis of the Inter-Blockchain  
Communication Protocol: Are Asset Transfer and Data Transport  
Semantically Unified, Stacked, or Conflicting?**

developed by

**Jazztine Kirvy Gonzales**

Bachelor of Science in Computer Science

**Jonel C. Ganalon**

Bachelor of Science in Computer Science

and submitted in partial fulfillment of the requirements of their respective Bachelor of Science degrees  
has been rigorously examined and recommended for approval and acceptance.

**First Panel Member**

Panel Member

Date signed: \_\_\_\_\_

**Second Panel Member**

Panel Member

Date signed: \_\_\_\_\_

**Third Panel Member**

Panel Member

Date signed: \_\_\_\_\_

**Adviser's Name**

Project Advisor

Date signed: \_\_\_\_\_

The Senior Project entitled

**Formal Specification and Behavioural Analysis of the Inter-Blockchain  
Communication Protocol: Are Asset Transfer and Data Transport  
Semantically Unified, Stacked, or Conflicting?**

developed by

**Jazztine Kirvy Gonzales**

Bachelor of Science in Computer Science

**Jonel C. Ganalon**

Bachelor of Science in Computer Science

and submitted in partial fulfillment of the requirements of their respective Bachelor of Science degrees  
is hereby approved and accepted by the Department of Computer Science, College of Computer  
Studies, Ateneo de Naga University.

**Lowie Vincent Bisana, MIT**

Chair, Department of Computer Science

Date signed: \_\_\_\_\_

**Joshua Martinez**

Dean, College of Computer Studies

Date signed: \_\_\_\_\_

# Declaration of Original Work

We declare that the Senior Project entitled

## **Formal Specification and Behavioural Analysis of the Inter-Blockchain Communication Protocol: Are Asset Transfer and Data Transport Semantically Unified, Stacked, or Conflicting?**

which we submitted to the faculty of the

### **Department of Computer Science, Ateneo de Naga University**

is our own work. To the best of our knowledge, it does not contain materials published or written by another person, except where due citation and acknowledgement is made in our senior project documentation. The contributions of other people whom we worked with to complete this senior project are explicitly cited and acknowledged in our senior project documentation.

We also declare that the intellectual content of this senior project is the product of our own work. We conceptualized, designed, encoded, and debugged the source code of the core programs in our senior project. The source code of third party APIs and library functions used in our program are explicitly cited and acknowledged in our senior project documentation. Also duly acknowledged are the assistance of others in minor details of editing and reproduction of the documentation.

In our honor, we declare that we did not pass off as our own the work done by another person. We are the only persons who encoded the source code of our software. We understand that we may get a failing mark if the source code of our program is in fact the work of another person.

**Jazztine Kirvy Gonzales**

4 - Bachelor of Science in Computer Science

**Jonel C. Ganalon**

4 - Bachelor of Science in Computer Science

This declaration is witnessed by:

**Adviser's Name**

Project Advisor

# Formal Specification and Behavioural Analysis of the Inter-Blockchain Communication Protocol: Are Asset Transfer and Data Transport Semantically Unified, Stacked, or Conflicting?

by

Jazztine Kirvy Gonzales, Jonel C. Ganalon

## ABSTRACT

The Inter-Blockchain Communication (IBC) protocol is the most widely adopted interoperability solution, enabling over 100 heterogeneous blockchains to exchange both digital assets and arbitrary data. However, a fundamental question remains unanswered: Are IBC’s asset-transfer (ICS-020) and data-transport (ICS-004) behaviours semantically unified, stacked, or conflicting? This research provides the first formal specification and behavioural analysis of IBC using process algebra and labelled transition systems (LTS). Unlike prior trace-based formalisations using  $TLA^+$ , process algebra preserves branching structure, concurrent composition, and silent abstraction which are essential for detecting behavioural anomalies. The research produces complete process algebra specifications of ICS-004 and ICS-020, extracts LTS models via structural operational semantics, and compares observable behaviours using weak bisimulation and deadlock analysis. The three analytical scenarios are operationally defined: unified (weak bisimilarity), stacked (trace inclusion without bisimilarity), or conflicting (emergent deadlocks under parallel composition). The methodology follows five phases: domain analysis, process algebra specification, LTS extraction, behavioural comparison, and reusable documentation. Findings will provide developers, auditors, and chain operators with mathematically grounded assurance of IBC’s semantic interoperability, while establishing a reusable formal framework for comparing other protocols such as Polkadot XCM and Axelar. This work addresses critical gaps in blockchain interoperability theory: no prior formal comparison of asset versus data behaviours, no process-algebraic treatment of IBC semantics, and no operational criteria for unified-stacked-conflicting classification.

We dedicate this research work to all of humanity.

# ACKNOWLEDGEMENTS

We thank everyone who helped me finish this thesis.

# TABLE OF CONTENTS

|          |                                                                              |           |
|----------|------------------------------------------------------------------------------|-----------|
| <b>1</b> | <b>Introduction</b>                                                          | <b>1</b>  |
| 1.1      | Project Context . . . . .                                                    | 2         |
| 1.2      | Purpose and Description . . . . .                                            | 3         |
| 1.3      | Objectives . . . . .                                                         | 4         |
| 1.4      | Scope and Limitations . . . . .                                              | 5         |
| <b>2</b> | <b>Review of Related Literature</b>                                          | <b>6</b>  |
| 2.1      | Evolution of Blockchain Interoperability . . . . .                           | 7         |
| 2.2      | The IBC Protocol: A Unified Approach to Interoperability . . . . .           | 12        |
| 2.2.1    | Protocol Architecture: Separation of Transport and Application Layer . . . . | 12        |
| 2.2.2    | Asset Transfer in IBC (ICS-020) . . . . .                                    | 12        |
| 2.2.3    | Arbitrary Data Transfer in IBC (ICS-004) . . . . .                           | 13        |
| 2.2.4    | Existing Formal Analysis of IBC . . . . .                                    | 13        |
| 2.2.5    | The Need for Process-Algebraic Analysis of IBC . . . . .                     | 14        |
| 2.3      | Formal Methods to Blockchain Interoperability . . . . .                      | 14        |
| 2.4      | Synthesis and Research Gap . . . . .                                         | 18        |
| <b>3</b> | <b>Theoretical Framework</b>                                                 | <b>21</b> |
| 3.1      | Systems and System Behaviour . . . . .                                       | 21        |
| 3.2      | Interoperability as System Composition . . . . .                             | 22        |
| 3.3      | Foundations of Process Algebra . . . . .                                     | 23        |
| 3.3.1    | Atomic Actions and Alphabet . . . . .                                        | 23        |
| 3.3.2    | Communication and Handshaking . . . . .                                      | 23        |



|          |                                                                          |           |
|----------|--------------------------------------------------------------------------|-----------|
| 3.3.3    | Process Operators . . . . .                                              | 23        |
| 3.3.4    | Encapsulation . . . . .                                                  | 24        |
| 3.3.5    | Recursion and Abstraction . . . . .                                      | 24        |
| 3.3.6    | Operation Semantics . . . . .                                            | 25        |
| 3.4      | Labelled Transition Systems as Semantic Model . . . . .                  | 25        |
| 3.5      | Behavioural Equivalences . . . . .                                       | 26        |
| 3.5.1    | Trace Equivalence (Partial and Completed) . . . . .                      | 26        |
| 3.5.2    | Bisimulation (Strong and Weak) . . . . .                                 | 27        |
| 3.5.3    | Branching Bisimulation and Cones and Foci . . . . .                      | 27        |
| 3.6      | From Process Algebra to LTS: Operational Semantics . . . . .             | 27        |
| 3.6.1    | Atomic Action . . . . .                                                  | 28        |
| 3.6.2    | Sequential Composition ( $P \cdot Q$ ) . . . . .                         | 28        |
| 3.6.3    | Alternative Composition ( $P + Q$ ) . . . . .                            | 28        |
| 3.6.4    | Parallel Composition ( $P \parallel Q$ ) . . . . .                       | 28        |
| 3.6.5    | Communication (Synchronisation) . . . . .                                | 28        |
| 3.6.6    | Encapsulation ( $\partial_H(P)$ ) . . . . .                              | 28        |
| 3.6.7    | Abstraction ( $\tau_I(P)$ ) . . . . .                                    | 28        |
| 3.7      | Application to Blockchain Interoperability . . . . .                     | 29        |
| 3.8      | Framework Synthesis . . . . .                                            | 29        |
| <b>4</b> | <b>Conceptual Framework and Methodology</b>                              | <b>30</b> |
| 4.1      | Conceptual Framework . . . . .                                           | 30        |
| 4.1.1    | Observable Behaviour in IBC . . . . .                                    | 31        |
| 4.1.2    | Trace Definition for IBC . . . . .                                       | 31        |
| 4.1.3    | Branching Structure in IBC . . . . .                                     | 32        |
| 4.1.4    | Deadlock in IBC . . . . .                                                | 32        |
| 4.2      | Research Paradigm: Formal Modelling as Inquiry . . . . .                 | 32        |
| 4.2.1    | The Three Analytical Scenarios (Unified, Stacked, Conflicting) . . . . . | 33        |
| 4.2.2    | Conceptual Framework Diagram . . . . .                                   | 34        |
| 4.3      | Research Methodology . . . . .                                           | 34        |
| 4.3.1    | Phase 1: Domain Analysis and Atomic Action Identification . . . . .      | 35        |
| 4.3.2    | Phase 2: Process Algebra Specification . . . . .                         | 37        |

|          |                                                  |           |
|----------|--------------------------------------------------|-----------|
| 4.3.3    | Phase 3: LTS Extraction . . . . .                | 39        |
| 4.3.4    | Phase 4: Behavioural Comparison . . . . .        | 40        |
| 4.3.5    | Phase 5: Documentation and Reusability . . . . . | 42        |
| 4.4      | Validation and Quality Criteria . . . . .        | 43        |
| 4.4.1    | Correctness of Formalisation . . . . .           | 44        |
| 4.4.2    | Completeness of LTS Extraction . . . . .         | 44        |
| 4.4.3    | Reproducibility . . . . .                        | 44        |
| 4.4.4    | Limitations . . . . .                            | 44        |
| 4.5      | Ethical Considerations . . . . .                 | 45        |
| 4.6      | Summary . . . . .                                | 45        |
| <b>A</b> | <b>Use of Generative AI</b>                      | <b>46</b> |
| A.1      | Brainstorming . . . . .                          | 46        |
| A.2      | Searching . . . . .                              | 46        |
| A.3      | Formatting . . . . .                             | 47        |
| A.4      | Debugging . . . . .                              | 47        |

# LIST OF FIGURES

4.1 Conceptual Framework for IBC Behavioural Analysis (Five Phases) . . . . . 34

# LIST OF TABLES

|     |                                                                         |    |
|-----|-------------------------------------------------------------------------|----|
| 2.1 | Survey and Protocol Design Literature . . . . .                         | 10 |
| 2.2 | Formal Methods Literature (Asset-Focused) . . . . .                     | 17 |
| 2.3 | Formal Methods Literature (Data-Focused and General) . . . . .          | 18 |
| 2.4 | Summary of Existing Work Against This Research's Key Criteria . . . . . | 19 |
| 4.1 | Comparison of Empirical and Formal Modelling Paradigms . . . . .        | 32 |
| 4.2 | Operational Definitions for Unified, Stacked, and Conflicting . . . . . | 33 |
| 4.3 | Process Algebra Notation Legend . . . . .                               | 42 |

# Chapter 1

## Introduction

After the inception of blockchain as a distributed ledger in 2009 with the invention of Bitcoin, the concept of **blockchain interoperability (BI)** soon followed and started to receive attention [7]. It is the ability of different blockchain networks to communicate and exchange data, or currency and assets without needing a centralised arbiter [37]. This is motivated by the realisation that blockchains have become isolated silos or systems incapable of communicating with their outside environment. Interoperability permits technology to develop beyond its limitation to scalability, reduces fragmentation in the ecosystem, and increases the use of liquidity in different platforms [9] [63]. Without this, innovation will remain limited as digital assets and information remain stagnant and isolated within their own blockchain networks.

The early protocols for BI started development within 2015-2016. They are mainly focused on the atomic exchange of digital asset or tokens between different networks [78]. These protocols, such as sidechains, notary schemes, and hashed time-lock contracts (HTLC), are categorised as **asset-based** since their original goal is to facilitate the exchange of value and cryptocurrency through an autonomous and synchronised way [73].

When the blockchain usage moved beyond cryptocurrency to complicated applications like provenance, tracking, and Internet of Things (IoT), new protocols aimed to handle arbitrary data evolved [38]. As data from many applications quickly grow in volume, the need to prioritise sharing arbitrary information and state transitions between networks has become more evident [73] [37]. Since the earlier blockchains are treated as isolated silos, the mere exchange of value is insufficient for sectors

like supply chain and healthcare that need wider coordination and sharing of information [78] [51]. This need led to the creation of cross-blockchain smart contracts and messaging frameworks allowing decentralised applications (dApps) to coordinate and verify information from different chain without undergoing a centralised arbiter [7].

Recognising the limitations of single-purpose protocols, developers created unified interoperability protocols capable of handling both assets and arbitrary data. The **Interblockchain (IBC)** protocol is the leading example of this. It handles both digital assets and arbitrary data by means of its payload-agnostic design; information is delivered as byte-encoded packets whatever it contains [19] [27]. This works by separating the transport layer (TAO) (that focuses on transportation, authentication, and arrangement of data) and the application layer (that gives meaning to each payload) [32].

For assets, IBC uses guidelines like (InterChain Standards) ICS-20 for locking the original asset and minting the corresponding vouchers in the target blockchain [61] [31] [27]. Meanwhile for arbitrary data and application logic, ICS-27 (Interchain Accounts) and ICS-31 (Interchain Queries) are used in order to execute commands or read states from other network without needing a centralised arbiter [61] [31]. IBC serves as a general-purpose communication layer similar to TCP/IP of the Internet that allows heterogeneous blockchains to exchange whatever type of information in a way that is safe and trust-minimised.

## 1.1 Project Context

The **Inter-Blockchain Communication (IBC) Protocol** is considered as the most used and widely adopted interoperability protocol that can handle both digital assets and arbitrary data [31] [7]. As a general-purpose message-passing protocol, it serves as the connective tissue of more than 100 different blockchains (where 107 chains uses reference implementation of IBC-Go) to cooperate in a way that is secure, permissionless, and trust-minimised. Its payload-agnostic design enables it to deliver whatever information that is encoded in bytes — from simple token transfers using ICS-20 standard to more complex arbitrary data and application logic like ICS-27 (Interchain Accounts) and ICS-31 (Interchain Queries). Because of its wide adoption and ability to serve as a universal standard like TCP/IP of Internet, it is the leading solution in creating an Internet of Blockchains that supports different workflows in the whole ecosystem [37].

However, despite IBC's widespread adoption, a fundamental question remains unanswered: Do IBC's asset-transfer and data-transfer behaviours share the same core semantics (unified), are assets semantics merely built on top of data transport without fundamental integration (staked), or do these behaviours interfere with each other in ways that create observable anomalies (conflicting)?

**Semantic interoperability**, or the ability of different blockchain networks to understand exchanged data and use it meaningfully in their own systems, depends on answering this question [63] [7]. Without a clear understanding of how asset and data behaviours relate within a unified protocol like IBC, developers cannot be certain that cross-chain operations will behave predictably. At the same time, auditors cannot guarantee that security properties hold across both asset transfers and arbitrary message passing.

**Formal modelling** answers the problem of semantic interoperability by providing a mathematical and clear description of the characteristics of the systems that removes ambiguity or misconceptions on how data should be understood and processed [70]. It serves as the foundation in ensuring that application-specific semantics are maintained amidst the differences in architecture and consensus mechanisms [63] [7]. Formal verification like model checking, provides irrefutable evidence to the developers and auditors that the coordination of the behaviours of assets and arbitrary data remain predictable and safe within a unified protocol like the IBC.

Currently, formalisation on IBC is focused on its TAO using tools like TLA+ in order to verify the correctness, safety, and liveness of its protocol [77]. There is a need to formalise its semantic interoperability layer because although ICS dictates guidelines for applications, there is insufficient mathematical foundation that verifies the coordination of the behaviours of asset and arbitrary data within system.

This research will use **Process Algebra** to model the behaviour of IBC protocol.

## 1.2 Purpose and Description

The main purpose of this research is to formally specify and analyse the observable behaviours of the IBC protocol using process algebra and labelled transition systems (LTS). Specifically, this research will:

1. Produce the first complete process algebra specification of IBC's core protocols (ICS-004 for data transport and ICS-020 for asset transfer).

2. Extract LTS behavioural models from these specifications.
3. Analyse whether IBC's asset-transfer and data-transfer behaviours are unified, stacked, or conflicting.

Unlike the prior formal analyses of IBC using TLA+ trace-based verification, this research uses process algebra which is a formalism uniquely suited to capturing concurrent interaction, branching structure, and silent abstraction. This provides a complementary perspective that can reveal behavioural anomalies that trace-based methods might miss.

Aside from developers and auditors, this will give dApp developers assurance and predictability in developing cross-chain applications. For chain operator and validator, this will reduce development risk and ensures stability of the system against dynamic runtime errors like deadlock or livelock [38] [24]. Lastly, end-users can ensure the integrity and security of their asset and transaction in a trust-minimised environment [38].

To guide this research, the following questions are posed:

1. How can IBC (including both asset transfer and data transfer) be formally specified using process algebra?
2. What observable behaviours (traces, ordering constraints, branching structure, deadlocks, synchronisation patterns) emerge from the LTS model of IBC?
3. Within the IBC specification, are the behavioural patterns for asset transfer and data transfer:
  - a. Unified (they share the same core semantics)?
  - b. Stacked (asset semantics are built on top of data transport without fundamental integration?)
  - c. Conflicting (they interfere with each other in ways that cause observable behavioural anomalies?)

### 1.3 Objectives

To achieve the main purpose of this research, the following specific objectives must be accomplished:

1. Identify and define the atomic actions, alphabets, communication ports, and data types necessary to model IBC's ICS-004 and ICS-020 in process algebra.
2. Formally specify IBC's core interaction patterns using process algebra operators.
3. Extract LTS models from the process algebra specifications, capturing all reachable states and observable transitions.



4. Analyse the LTS models to identify traces, ordering constraints, branching structures, deadlock states, and synchronisation pattern for both asset-transfer and data-transfer scenarios.
5. Compare the behavioural patterns for asset transfer and data transfer within IBC specification to determine whether they are unified, stacked, or conflicting.
6. Document the process algebra specification and LTS models in a reusable form suitable for future comparative studies with other interoperability protocols like Polkadot XCM and Axelar.

## 1.4 Scope and Limitations

The formal model abstracts away low-level details such as cryptographic verification (Merkle proofs), light client synchronisation, and relayer behaviour

Economic incentives (gas costs, slashing, liquidity constraints) are not modelled

The analysis focuses on observable behaviour only; security properties beyond what is observable (e.g., double-spending resistance) are not verified

Only IBC's core ICS-004 and ICS-020 standards are modelled; other application standards (ICS-27, ICS-31) are excluded

No empirical validation or actual protocol instance execution is performed

## Chapter 2

# Review of Related Literature

In the current landscape of blockchain technology, the emergence of heterogeneous networks has revealed the term "information isolated islands" or "value silos" [73] [11] [80]. Although individual blockchains proved to be useful as independent ledgers, the lack of communication between them limits their potential widespread adoption [30] [16] [49]. Blockchain interoperability then becomes a critical interaction between distributed systems [73] [44] [49].

Blockchain systems have been described as an independent state machine with its own consensus rules and encryption standards [11] [7]. Because they do not have a shared state or global clock between them, their interactions must happen asynchronously; they must instead use message passing and state synchronisation [73] [77]. In other words, blockchain interoperability is a process wherein one source chain is trying to change the state of a target chain by means of delivering information or assets under certain security assumptions [73] [74] [8].

Most of study in the literature focus on the blockchain interoperability architecture. That is, whether it uses relays, sidechains, or notaries [39] [14] [43] [15]. Nevertheless, recent studies have put emphasis that architecture only gives structure and real security and effectiveness of interoperability depends on the behaviour of the system [47] [46] [49]. This behaviour dictates how systems guarantee ACID (atomicity, consistency, isolation, durability) properties in an asynchronous network [73] [11] [74] [44].

This means that the sequencing of state transitions, timing of cryptographic proofs, and reactions to failure or timeout reveals the system's behaviour [58] [17] [23] [77]. The analysis of behaviours,

instead of the static design, gives deeper understanding on how different components (such as smart contracts and relayers) interact during different processes. That is so to achieve semantic interoperability where the meaning and context of data or asset from the source chain gets accurately interpreted and valid to the target chain [6] [65] [44].

## 2.1 Evolution of Blockchain Interoperability

Surveying the literature, it is noticeable that the classifications of interoperability reflects the underlying logic on how to trust and treat information between blockchains [73] [59] [75]. Although categorical labels like notary schemes, sidechains/relays, and hash-locking are usually used, there is a deeper abstraction that surfaces in recent literature. That is, the perspective of looking at blockchain interoperability as a multi-layered problem derived from the OSI model of the internet [30] [23].

The common abstraction from most of the sources is the **source-target model**. In this model, an operation from the source chain needs to trigger a corresponding state change to the target chain [23] [16]. The following differentiates a critical difference on how this interaction gets seen:

- **Interoperability as Transfer:** In this perspective, the logic focuses on migration or transfer of ownership. Every transaction must follow all-or-nothing atomicity in order to avoid double-spending [73] [45] [23]. Asset-based protocols belong here, ensuring that the value will not vanish or gets duplicated between different silos [6] [75].
- **Interoperability as Communication:** In this perspective, the system gets treated as a **message-oriented service** [73] [39] [35]. The goal is not to transfer the content of the source chain but to invoke a functionality in the target chain using arbitrary data [59] [7]. This is the logic of the data-based protocols like Arbitrary Message Passing (AMP) [4].

It is critical to make the distinction between asset-based and data-based interoperability because they usually remain implicit in a number of older literature. For instance, in the classical SoK (Systematisation of Knowledge) of Buterin, the focus was always in value exchange or value islands [73] [1] [59]. However, in the newer frameworks, there is an explicit division in the domain between **Asset Interoperability** (migration/exchange) and **Data Interoperability** (cross-chain calls/integration) [23] [16] [44]. This gap is important as a number of current models are designed

for assets but gets applied to data or vice-versa without taking each different semantic requirements into consideration [6] [44].

This differeng individual trust boundary and security assumptions of each approaches contributes to the fragmented landscape of blockchain interoperability[6]. Although there is an attemot to have a Generic Interoperability Paradigm comprising of Setup, Commit, Verify, and Abort phases, the literature has not fully explained how these phases differ in their **observable behaviour** when assets are passed instead of arbitrary data [23] [44]. This forms the need of a **comparative semantic analysis** to see if these models are enough to encompass the behaviour of both asset and data exchanges [44].

- **Asset-Based Interoperability**

The literature treats asset-based interoperability not solely as a tool to transfer currency but also as a **closed system of value synchronisation** [59] [16]. The fundamental logic of these protocols is the preservation of the INTEGRITY of assets while they are moving between different ledgers [45] [59] [6].

As a system, asset-based interoperability comprises the following interdependent components: **Chains (Source and Target):** These serve as state machines that record ownership [73] [23] [16]. **Smart Contracts (Custodian vs. Debt Issuer):** In the source chain, there is usually a custodian contract (vault) that locks asset; while in the target chain, there is a debt issuer (money printer) that makes synthetic representation or wrapped tokens [57] [66]. **Users and Intermediaries:** Included here is the initiator (user), relayers that transfer proofs, and attestors that verify if the lock event is valid before doing the minting [57] [47] [6].

This shows that asset-based interoperability is a content-aware system. Meaning, it is necessary that the protocol knows the specific semantics of asset (e.g. ERC-20 standard) in order to ensure that the minted token is semantically equal to the original asset [47] [59] [23].

The observable behaviour of asset-based protocols follow a strict sequential execution flow. Based on existing formal models (such as Event-B analysis), the usual sequence follows: **Lock/Burn (Source):** Immobilisation of the original value [57] [47] [23]. **Verification:** Passing the Merkle proof or SPV (Simplified Payment Verification) proceeding to the target chain [73] [50] [11] [23]. **Mint/Claim:** Producing of an equal value at the target chain [47] [64] [44].

This shows that there is happens-before relationship. The mint transaction at the target chain has causal dependency on the lock transaction at the source chain [47] . If this sequence is not

followed, there is a risk of atomicity failure where there is a possibility of minting assets that are not locked; resulting to double-spending [45] [6] [23] [43].

The major invariant that safeguards this system is the Global Supply Invariant. This refers to maintaining a constant value to the overall count of asset units in all networks [59] [43] [11].

It is critical that the safety (no bad behaviour or duplication) of asset-based protocols gets prioritised over the liveness (fast termination) [47] [11]. Because of this, the observable behaviour has long confirmation periods (to wait the finality from the source chain) and competition periods (for fraud proofs) [23] [73] [74] [52]. These delays are not just performance issue but a behavioural necessity in order to protect the atomicity of the system [17] [50] [16].

Nevertheless, this behaviour results to capital inefficiency [66] [56] [44]. That is because of the locking of assets, the liquidity gets trapped in the contracts, that serves as a big limitation that is not usually seen in data-based approaches [66] [23] [4]. This creates the gap in the literature where even though the safety of asset transfers can be formally modelled, there is insufficient comparison if this strict ordering is also needed for arbitrary data exchange [47] [46].

#### • Data-Based Interoperability

Unlike asset-based interoperability where it is illustrated as a mechanism of changing ownership, the literature describes data-based interoperability as a decentralised communication system [50] [11] [39]. If asset-based focuses on liquidity, data-based on the other hand focuses on the arbitrary message passing (AMP). This is a mechanism where the system acts as a transport layer for whatever type of information [22] [4].

Being a communication system, its architecture comprises of the following interdependent components: **Chains and Modules:** Every blockchain hosts modules or handlers (smart contracts). They serve as entry and exit points of data [22] [17]. **Relayers and Connectors:** Since blockchains are isolated islands and incapable of sending data natively, they need off-chain relayers (such as Hermes) that scans blocks for pending messages and transfer them to the target chain [22] [17]. **Channels and Connections:** These are the stateful abstractions that maintains logical link between two chains that gives ordering and authentication to packets [11] [? ].

The literature sees the system as a payload-agnostic service. This shows that the protocols do not care whether what is inside the packets is a token transfer or a complex smart contract call. Its purpose is to ensure that the opaque bytes are delivered with integrity [39] [22] [17] [11].

The observable behaviour of data-based interoperability is guided by handshake protocols and asynchronous execution [50] [11] [77]. **Handshake Process:** This is unlike the simple lock-and-mint; data-based systems follows a strict 4-step handshake (OpenInit -> OpenTry -> OpenAck -> OpenConfirm) in order to establish a connection [11] [77]. This behaviour is needed to ensure that both chains are ready to accept data under the same security assumptions [77]. **Asynchronous Message Passing:** The interaction here is non-blocking. The source chain delivers a message and can proceed to other operations while the packet is in-flight [50] [11]. The success of the interaction depends on Acknowledgements (ACK) or Timeouts. They are observable events that dictates if an operation is deemed Delivered or Expired [17] [77].

The fundamental difference between the two is rooted on their interaction logic: **State-Dependent vs. Instruction-Dependent:** Unlike the asset-based that depends on state provenance (it needs the verification of the lock state before minting), data-based behaviour is instruction-dependent [76] [23]. Its aim is to trigger a functionality in the target system that will work under its own rules [73] [7]. **Atomicity vs. Eventual Delivery:** In asset transfers, atomicity is commonly all-or-nothing under a capital constraint [59] [6]. Meanwhile, in data exchange, it prioritises liveness and delivery guarantees. It means that every message should have a definite result (accepted or timeout) inside the asynchronous network [11] [59] [77].

The behaviour here depends on the reliability of the transport layer. Nonetheless, the critique in the current literature is the dependency of these systems to untrusted layers [77] [22]. Formal analyses (such as the TLA+) shows that even the standardized protocols can have trapped packets or incomplete handshakes if the relayers' behaviour cannot be properly modelled [77]. This pushed the need for deeper comparative semantic analysis to see how it is possible to converge the safety of assets and the liveness of data in one behavioural model.

Table 2.1: Survey and Protocol Design Literature

| Citation | Type of Work                     | Key Limitation                                       |
|----------|----------------------------------|------------------------------------------------------|
| [66]     | Survey / Protocol Design (Asset) | No formal model; no behavioural analysis             |
| [57]     | Survey / SoK (Hybrid bridges)    | Lacks formal system model; no concurrency semantics  |
| [14]     | Survey (Cross-chain general)     | Architectural focus only; no behavioural equivalence |
| [4]      | Framework / Dataset (Data/AMP)   | No process-algebraic specification                   |

| Citation | Type of Work                                | Key Limitation                                          |
|----------|---------------------------------------------|---------------------------------------------------------|
| [44]     | Survey (Cross-chain general)                | No formal comparison of asset vs data behaviours        |
| [22]     | Protocol Analysis (Data/IBC)                | No formal semantics; no LTS extraction                  |
| [37]     | Survey (Cross-chain general)                | Lacks behavioural criteria for unification/stacking     |
| [23]     | Survey (Asset + Data)                       | Identifies asset/data distinction, no formal comparison |
| [1]      | Survey (General)                            | No concurrency or bisimulation                          |
| [30]     | SoK (Layered architecture)                  | No process algebra; static taxonomy only                |
| [40]     | Protocol Design (Cross-chain transactions)  | No formal verification of behaviour                     |
| [11]     | Survey (Data)                               | No behavioural semantics for handshake protocols        |
| [16]     | Protocol Design (Asset + Data)              | Security assumptions only; no behavioural model         |
| [65]     | Framework (Semantic interoperability)       | No formal concurrency semantics                         |
| [56]     | Protocol Design (Cross-chain communication) | No formal verification                                  |
| [42]     | Survey (Data/logistics)                     | Domain-specific; no general behavioural theory          |
| [3]      | Protocol (Asset)                            | No formal semantics                                     |
| [15]     | Protocol Design (Asset/notary)              | No behavioural equivalence                              |
| [43]     | Survey (Cross-chain technology)             | No formal comparison                                    |
| [75]     | Systematic Survey (Asset + Data)            | Identifies gap but no formal solution                   |
| [6]      | Survey / Gap Analysis (Asset + Data)        | Calls for formal methods but does not apply PA          |
| [80]     | Protocol Design (Asset/notary + signatures) | No LTS or bisimulation                                  |
| [29]     | Survey (Cross-chain challenges)             | High-level only                                         |
| [39]     | Survey (Heterogeneous communication)        | No formal behaviour model                               |
| [52]     | Survey (Cross-chain technology)             | No concurrency semantics                                |
| [76]     | Protocol Design (Asset + Data)              | No formal verification                                  |
| [64]     | Protocol (Asset)                            | No behavioural equivalence                              |
| [35]     | Survey (Interoperability landscape)         | No formal methods applied                               |
| [58]     | Survey (Cross-chain mechanisms)             | No process algebra                                      |
| [59]     | Framework (Asset/burn-to-claim)             | No LTS or branching behaviour                           |
| [49]     | Survey (Interoperability role)              | Behavioural analysis missing                            |
| [7]      | Survey (Past, present, future trends)       | No formal semantics                                     |
| [73]     | SoK (Asset + Data)                          | Calls for behavioural semantics but does not provide    |

## 2.2 The IBC Protocol: A Unified Approach to Interoperability

While the previous sections have treated asset-based and data-based interoperability as conceptually distinct categories, the Inter-Blockchain Communication (IBC) protocol represents an attempt to unify both within a single framework. Developed by the Cosmos ecosystem, IBC has become the most widely adopted interoperability protocol, connecting over 100 blockchains as of 2025 [31] [7].

### 2.2.1 Protocol Architecture: Separation of Transport and Application Layer

IBC is designed as a general-purpose message-passing protocol analogous to TCP/IP for the Internet. Its architecture explicitly separates two layers [19] [27]:

- **Transport, Authentication, and Ordering (TAO) Layer (ICS-004):** This layer handles the low-level concerns of packet delivery: establishing connections via a 4-step handshake (OpenInit → OpenTry → OpenAck → OpenConfirm), ordering packets, timeouts, and acknowledgements. The TAO is payload-agnostic – it treats all data as opaque bytes [77] [32].
- **Application Layer (ICS-020, ICS-027, ICS-031):** This layer gives meaning to the payload. ICS-020 defines fungible token transfers (asset interoperability). ICS-027 defines Interchain Accounts (allowing one chain to control an account on another). ICS-031 defines Interchain Queries (reading state from another chain). The separation allows the same TAO to support both asset and data semantics [61] [31].

### 2.2.2 Asset Transfer in IBC (ICS-020)

Under ICS-020, asset transfer follows a **lock-and-mint** pattern consistent with asset-based interoperability described earlier:

1. **Lock/Burn on Source Chain:** The source chain locks the original tokens in a module account or burns them.
2. **Packet Transmission:** The lock event is packaged into an IBC packet and sent via the TAO layer.



3. **Mint on Target Chain:** Upon packet receipt and proof verification, the target chain mints an equivalent amount of vouchers (typically prefixed with the source chain's channel identifier).
4. **Reverse Path (Transfer Back):** The same process in reverse burns vouchers and unlocks original tokens.

This behaviour preserves the **Global Supply Invariant** or the total supply across both chains remains constant [59] [43].

### 2.2.3 Arbitrary Data Transfer in IBC (ICS-004)

At the TAO layer, the protocol exhibits data-based interoperability semantics:

- **Asynchronous Packet Delivery:** Source chain commits a packet; target chain receives it independently. The source chain may continue other operations while the packet is in-flight [50].
- **Acknowledgement and Timeout:** Every packet must eventually result in either an acknowledgement (successful receipt and processing) or a timeout (failure to receive within a deadline). The alternative composition ‘ack + timeout’ creates observable branching behaviour [17] [77].
- **Relayer-Mediated Forwarding:** Off-chain relayers scan both chains, detect packet commitments, and submit proofs to the target chain. Relayer behaviour is internal to the protocol – from an external observer's perspective, only packet send and receive/ack events are visible [22].

### 2.2.4 Existing Formal Analysis of IBC

To date, formal verification of IBC has focused primarily on the TAO layer using trace-based methods: **Wei et al. (2023)** modelled IBC's TAO in TLA+ and discovered two critical design-level issues: (1) **incomplete handshakes due to identifier exhaustion** where channels could enter a state where no further handshake steps were possible; (2) **trapped packets** where packets could neither be received nor timed out under certain relayer conditions [77]. While this analysis confirmed safety and liveness properties of packet delivery, it left several questions unanswered from a **behavioural semantics** perspective:

- Does the asset transfer behaviour (ICS-020) share the same core semantics as the data transport behaviour (ICS-004), or are assets simply stacked on top of data with additional constraints?

- Do asset and data behaviours interfere with each other when both flow through the same channels?
- What branching structures (e.g., timeout vs acknowledgement choices) characterise each behaviour, and are they equivalent?

These questions require a **behavioural** rather than trace-based analysis. A behavioural analysis preserves the branching structure of the system and can compare processes using bisimulation equivalence.

### 2.2.5 The Need for Process-Algebraic Analysis of IBC

TLA+ verification, while valuable for detecting linear temporal logic violations, models behaviour as sequences of states (traces). As will be established in the theoretical framework, trace equivalence collapses branching structure: ‘ack+timeout’ (choice) is indistinguishable from ‘ack.timeout’ (sequence) under trace equivalence [26].

To answer whether IBC's asset and data behaviours are **unified**, **stacked**, or **conflicting**, a process-algebraic approach is required. Process algebra can:

1. Model the concurrent composition of asset and data flows using the parallel operator
2. Preserve branching structure using alternative composition
3. Abstract internal relay behaviour using the silent action
4. Compare behaviours using weak bisimulation rather than trace equivalence

This research provides the first process algebra specification of IBC's ICS-004 and ICS-020 standards, filling the gap identified above.

## 2.3 Formal Methods to Blockchain Interoperability

In the study of interoperability protocols, simple architecture diagrams alone are insufficient to guarantee the security and correctness of systems. Formal Methods surfaces in the literature as critical tools to avoid design flaws that can lead to large financial losses [77] [46]. In surveying the literature, formalisations of blockchain interoperability protocols have been developed using a variety of methodologies such as interactive theorem proving, model checking, and symbolic logic.

They generally focus on ensuring atomicity, safety, liveness, and consistency across heterogeneous networks.

- **Mechanised Proofs and Refinement (Isabelle/HOL, Coq, Event-B)** Isabelle/HOL has been used to provide rigorous, high-assurance proofs for several interoperability frameworks.
  - **Hierarchical Transactions (Canton):** The Canton interoperability protocol's transactions were formalised as authenticated data structures using a modular "Merkle functor" pattern [48]. This approach allows for defining inclusion proofs and merging them into multi-proofs while maintaining the data integrity.
  - **Global Consistency in IBC:** The IBC protocol was formalised as an instantiation of the Isabelle Infrastructure framework [34]. This model established a "global inconsistency" property, proving that scanning and submitting transactions between blockchains does not introduce inconsistencies across their respective ledgers.
  - **Fail-Safe Cross-Chain Bridges:** A formal model of the Sonic Gateway bridge was verified to be safe using a stepwise refinement approach [53]. This verification proved that even if the receiver chain fails (a "dead bridge" scenario), users can still retrieve their assets on the sender chain via specific recovery transactions.
  - **Cross-Domain Regulatory State Preservation:** A formal proof was established for regulatory state transitions (e.g., active or frozen status) across multiple domains [36]. It verified safety (bidirectional roundtrip preservation) and liveness (deterministic resolution of conflicting actions under Byzantine faults).
  - **Bitcoin Validation Procedure:** The first formal model of Bitcoin's transaction data structures and validation logic was mechanised in Coq [62]. It provided machine-verified proofs for essential properties like the impossibility of double-spending and the uniqueness of transactions.
  - **Inter-Contract Communication:** Coq has also been used to construct models that allow for interactions between contracts on different types of blockchains (e.g., depth-first vs. breadth-first execution models) [34].
  - **Gateway-Based Interoperability:** A gateway-based interoperability solution between Ethereum and Hyperledger Fabric was formalised in Event-B using the Rodin platform [46]. It automatically discharged 23 proof obligations (17 invariant preservation, 6 well-definedness) using Rodin's provers, confirming functional correctness and safety proper-

ties.

- **Temporal Logic of Actions (TLA+) and Model Checking** TLA+ and the TLC model checker have been utilised to identify functional and timing-related vulnerabilities in inter-chain protocols.
  - **IBC TAO Layer:** The TAO layer of IBC was modelled in TLA+ [77] (see the preceeding section on IBC Protocol).
  - **Tendermint Synchronisation (Fastsync):** The Fastsync protocol was formalised at multiple levels (high-level, low-level, and concurrency) in TLA+ [13]. Model checking helped refine properties related to termination and synchronisation, revealing that a global timeout might terminate the process before the node reaches the maximum height of its correct peers.
  - **Cross-Chain Swaps:** Other efforts have included TLA+ formal proofs for specific cross-chain swap protocols and the "burn-to-claim" asset transfer protocol [77].
- **Symbolic Logic and Tamarin**
  - **Hash Time Lock Contracts (HTLC):** The security properties of HTLCs, the most common mechanism for atomic cross-chain swaps, were formally analysed using the Tamarin prover [12]. This formalisation identified a critical "hidden" assumption: for the protocol to be secure, the growth speeds of the two blockchains involved must remain stable.
- **Timed Automata and UPPAAL**
  - **Pub-Sub Interoperability Protocol:** This protocol, which establishes communication between Hyperledger Fabric and Besu networks via a broker, was formalised as stochastic timed automata (STA) [2]. Using UPPAAL-SMC, researchers verified real-time properties and provided probabilistic estimations for message delivery, revealing an 8% probability of failure when the broker's throughput is significantly higher than that of the publishers and subscribers.
- **Abstract and Unified Frameworks**
  - **Atomic Multi-Chain Transaction Protocol:** Researches defined a uniform communication interface and a protocol inspired by two-phase commits to handle general transac-

tions spanning several chains [50]. This was supported by formal modelling of blockchains as **abstract state machines** to prove atomicity.

- **SmartML and WhyISon:** Modelling languages like SmartML provide a platform-independent framework for analysing smart contracts [71]. Similarly, WhyISon translates Michelson (Tezos) smart contracts into WhyML for automated deductive verification in the Why3 framework [20].
- **QLink Architecture:** A quantum-safe bridge architecture was modelled mathematically to link cryptographic (Post-Quantum Cryptography) and physical-layer (Quantum Key Distribution) security [21].

#### Comparative Analysis: Process Algebra vs. Existing Formalisms

In contrast to the state-based and trace-based approaches above, **Process Algebras** (e.g., ACP, CSP, pi-calculus) offer a compositionally rich behavioural perspective on interoperability. Below is a summary of what each surveyed method lacks.

Table 2.2: Formal Methods Literature (Asset-Focused)

| Citation | Formal Method Used    | Key Limitation                                  |
|----------|-----------------------|-------------------------------------------------|
| [47]     | Event-B               | State-based only; no concurrency or branching   |
| [45]     | Event-B               | Lacks liveness and asynchronous communication   |
| [46]     | Event-B               | No dynamic process creation; no mobility        |
| [48]     | Isabelle/HOL          | No concurrency or branching behaviour           |
| [53]     | Event-B refinement    | State-based; no concurrency semantics           |
| [62]     | Coq                   | Single ledger; not interoperability-focused     |
| [12]     | Tamarin               | No true concurrency or liveness for data        |
| [21]     | Mathematical model    | No process algebra; no bisimulation             |
| [72]     | CSP (Process Algebra) | Single protocol; no comparison of asset vs data |

Table 2.3: Formal Methods Literature (Data-Focused and General)

| Citation | Formal Method Used      | Key Limitation                                 |
|----------|-------------------------|------------------------------------------------|
| [77]     | TLA+                    | Trace-based only; no bisimulation or branching |
| [50]     | Abstract State Machines | No process algebra; no bisimulation            |
| [34]     | Isabelle/HOL            | No process algebra; trace-based only           |
| [36]     | Isabelle/HOL            | Safety/liveness but no branching equivalence   |
| [13]     | TLA+                    | Not IBC-specific; no asset-data comparison     |
| [2]      | Timed Automata (UPPAAL) | Probabilistic; no behavioural refinement laws  |
| [20]     | Why3/WhyML              | Single chain; not interoperability             |
| [17]     | Empirical / Measurement | No formal specification                        |

This gap motivates the need for a unified behavioural semantics — one that treats both asset and data interoperability as observable interaction patterns rather than isolated state transitions.

## 2.4 Synthesis and Research Gap

In surveying the literature, a clear pattern surfaces: although the improvement in the technical solutions to blockchain interoperability is growing fast, its theoretical and formal foundation still remains slow and fragmented. The following lay outs the critical gaps that reasons the need for this research.

1. **Gap of comparison:** No formal comparison of asset vs data behaviours within a single protocol.
2. **Gap of method:** No use of Process Algebra/LTS for interoperability semantics.
3. **Gap of behavioural criteria:** No operational definition of "unified/stacked/conflicting" in terms of observable behaviour.
4. **Gap of IBC semantics:** No process-algebra model of IBC that separates transport vs application behaviour.

This research fills these gaps by providing the first process algebra specification of IBC's core interoperability standards (ICS-004, ICS-020), extracting LTS models, and analysing whether asset

and data behaviours are unified, stacked, or conflicting.

Table 2.4: Summary of Existing Work Against This Research's Key Criteria

| Reference | Formal Method  | Asset-Data Comparison? | Uses Bisimulation? | Models IBC?       |
|-----------|----------------|------------------------|--------------------|-------------------|
| [73]      | None           | No (calls for it)      | No                 | No                |
| [77]      | TLA+           | No                     | No                 | ✓ (TAO only)      |
| [47]      | Event-B        | No                     | No                 | No                |
| [46]      | Event-B        | No                     | No                 | No                |
| [45]      | Event-B        | No                     | No                 | No                |
| [72]      | CSP            | No (asset only)        | Yes                | No                |
| [50]      | ASM            | No                     | No                 | No                |
| [34]      | Isabelle/HOL   | No                     | No                 | Yes (consistency) |
| [48]      | Isabelle/HOL   | No                     | No                 | No                |
| [53]      | Event-B        | No                     | No                 | No                |
| [36]      | Isabelle/HOL   | No                     | No                 | No                |
| [62]      | Coq            | No                     | No                 | No                |
| [12]      | Tamarin        | No                     | No                 | No                |
| [13]      | TLA+           | No                     | No                 | No                |
| [2]       | Timed Automata | No                     | No                 | No                |
| [21]      | Mathematical   | No                     | No                 | No                |
| [23]      | None           | Yes (identifies gap)   | No                 | No                |
| [6]       | None           | Yes (calls for formal) | No                 | No                |
| [44]      | None           | Yes (calls for formal) | No                 | No                |
| [30]      | None           | No                     | No                 | No                |
| [11]      | None           | No                     | No                 | No                |
| [17]      | Empirical      | No                     | No                 | Yes (performance) |
| [22]      | Empirical      | No                     | No                 | Yes (empirical)   |
| [66]      | None           | No                     | No                 | No                |
| [57]      | None           | No                     | No                 | No                |

| Reference            | Formal Method | Asset-Data Comparison? | Uses Bisimulation? | Models IBC? |
|----------------------|---------------|------------------------|--------------------|-------------|
| [3]                  | None          | No                     | No                 | No          |
| [59]                 | None          | No                     | No                 | No          |
| [16]                 | None          | No                     | No                 | No          |
| [39]                 | None          | No                     | No                 | No          |
| [43]                 | None          | No                     | No                 | No          |
| [75]                 | None          | No                     | No                 | No          |
| [80]                 | None          | No                     | No                 | No          |
| [15]                 | None          | No                     | No                 | No          |
| [64]                 | None          | No                     | No                 | No          |
| [76]                 | None          | No                     | No                 | No          |
| [56]                 | None          | No                     | No                 | No          |
| [40]                 | None          | No                     | No                 | No          |
| [14]                 | None          | No                     | No                 | No          |
| [4]                  | None          | No                     | No                 | No          |
| [37]                 | None          | No                     | No                 | No          |
| [1]                  | None          | No                     | No                 | No          |
| [65]                 | None          | No                     | No                 | No          |
| [42]                 | None          | No                     | No                 | No          |
| [29]                 | None          | No                     | No                 | No          |
| [52]                 | None          | No                     | No                 | No          |
| [58]                 | None          | No                     | No                 | No          |
| [35]                 | None          | No                     | No                 | No          |
| [49]                 | None          | No                     | No                 | No          |
| [7]                  | None          | No                     | No                 | No          |
| [20]                 | Why3/WhyML    | No                     | No                 | No          |
| <b>This research</b> | <b>ACP</b>    | <b>Yes</b>             | <b>Yes</b>         | <b>Yes</b>  |



## Chapter 3

# Theoretical Framework

This section provides the mathematical vocabulary and analytical tools necessary in answering the three research questions guiding this research. Specifically, it establishes: (1) how to formally specify IBC's atomic actions and processes, (2) how to extract observable behaviours including traces, deadlocks, and branching structures from such specifications, (3) how to operationally determine whether asset and data behaviours within IBC are unified, stacked, or conflicting using behavioural equivalence.

### 3.1 Systems and System Behaviour

A **system** is formally composed of two major aspects: process and data. In this framework, processes serve as control mechanisms that runs and manipulates data. Meanwhile, data remains static and passively comprises the system [25]. The composition of these elements forms a structure that can perform operations according to specified rules [60].

The **behaviour** of a system refers to the sum of its ability to interact or communicate. A behaviour is **observable** because it can be seen from its outside operations; particularly, the interaction of the inputs it accepts and the outputs it produces [55]. Every elementary step or operation that happens in the system is called **transition** that changes the system from its current configuration to another state [60].

A **Blockchain** is an example of a system that can be treated as a transaction-based **state machine** [79]. In this view, the blockchain is not just a storage of information but a dynamic mech-

anism where transactions serve as inputs that result to transitions of the distributed ledger's state. Every succesful transaction creates new state that is formally accepted by the whole network. The usage of **formal modelling** is critical in this topic in order to give sufficient mathematical evidence for the security and stability of interoperability systems. Formal modelling removes ambiguity and ensures the correctness in comparing asset-based and data-based mechanisms under one theoretical framework [24] [25].

### 3.2 Interoperability as System Composition

In process theory, interoperability can be formally described as a composition of two or more systems that work together to achieve a particular goal [41]. Under the framework of Process Algebra, this communication is symbolised by the parallel composition

$$(P||Q)$$

, wherein every blockchain network is treated as separate process that runs concurrently and independently from the other networks but with the ability to communicate to each other.

The communication inside this composition is run by the mechanism of **synchronisation** [55]. By means of a formal **communication function**, the individual actions of every blockchain gets to communicate. For example, the action that delivers the message (*send*) from a system needs to match the action of receiving (*read*) at the other system in order to complete a succcessful communication.

The behaviour of the formed composition is treated as observable from the perspective of an outsider viewer [55]. Using the **abstraction operator** the complex details of the internal communication between blockchains are hidden or made as *silent steps* ( $\tau$ ), so that what only remains are the external input-output relation of the whole inter-operational system [25]. The resulting traces or series observable actions serve as the evidence if the operation of the composed systems remain consistent to the original semantic specification.

This framework of formal composition gives way to formally analyse and compare the two ways of blockchain communications: the asset-based and data-based interoperability.

### 3.3 Foundations of Process Algebra

**Process Algebra** serve as the main theoretical instrument of this research particularly the principles of communication, composition, and abstraction. The research will use it to formally analyse and manipulate operations of blockchains. This is illustrated as the study of concurrent systems or communicating systems inside an axiomatic and algebraic framework [10] [25]. Instead of just using visual diagrams, processes are embodied as formal *process terms* using a mathematical language that allows linear notation and mechanised manipulation [25].

#### 3.3.1 Atomic Actions and Alphabet

System behaviour is constructed from a formally defined alphabet of atomic actions. Let:

$D$  : set of data

$P$  : set of communication ports

Atomic actions are defined as:

$Send : s_p(d)$

$Read : r_p(d)$

where  $d \in D$  and  $p \in P$  [68].

#### 3.3.2 Communication and Handshaking

Communication between processes is governed by a communication function:

$$\gamma(s_p(d), r_p(d)) = c_p(d)$$

This formalises the handshaking paradigm where communication happens only through synchronisation between matching send and read actions. Isolated actions are disallowed to ensure correctness in system interaction [68].

#### 3.3.3 Process Operators

In this framework, every system's process is composed of atomic actions (actions that has no duration) combined using the following operators: **Sequential Composition** ( $x \cdot y$ ):  $x$  is performed first

before  $y$ . **Alternative Composition** ( $x + y$ ): symbolises the nondeterministic choice between  $x$  and  $y$  operations. **Parallel Composition** ( $x || y$ ): symbolises the simultaneous execution of actions  $x$  and  $y$ .

A fundamental feature of process algebra is its **compositionality** wherein a large system can be treated as combined smaller and simpler parts [25]. Using the **parallel operator** ( $||$ ), individual blockchain network or protocol components can be modelled separately and then regrouped to see the whole behaviour of the inter-operational system [25]. This modular technique allows the analysis of the connection between asset-based and data-based protocols as a series of concurrent processes [25].

### 3.3.4 Encapsulation

To control system visibility: **Encapsulation** ( $\partial_H$ ) blocks unmatched internal actions. It is used in order to ensure these internal actions do not happen alone but as a part of a synchronized communication that causes transition to the state of the whole composition.

### 3.3.5 Recursion and Abstraction

Recursion allows specification of infinite or repetitive behaviours using fixed-point operators. For blockchain protocols, recursion models continuous channel operation:

$$Channel = Open \cdot (Send \cdot Receive + Timeout) \cdot Channel$$

The **silent action** ( $\tau$ ) represents internal, unobservable steps. Relay forwarding, proof verification, and internal state updates are modelled as  $\tau$  actions. That is, they affect the system's internal state but are not visible to an external observer.

The **abstraction operator**  $\tau_I$  renames all actions in set  $I$  to  $\tau$ , hiding implementation details. For IBC:

$$\tau_{\{forward\_packet, verify\_proof\}}(P_{IBC})$$

makes only packet sends, receives, acknowledgements, and timeouts observable. Meanwhile, internal relay behaviour becomes silent steps.

Key properties of  $\tau$ :

- $\tau \cdot P = P$  (silent steps can be removed without changing observable behaviour)
- $a \cdot (\tau \cdot (b + c))$  is observationally equivalent to  $a \cdot (b + c)$

### 3.3.6 Operation Semantics

The meaning of process expressions is formally defined using **Structural Operational Semantics (SOS)**. Transition rules specify how processes evolve. This ensures that every behaviour is derivable and mathematically well-defined [68].

## 3.4 Labelled Transition Systems as Semantic Model

Labelled Transition System (LTS) is a fundamental mathematical structure used to model computations and dynamic behaviour of systems [69]. A Labelled Transition System (LTS) is formally defined as a tuple  $(S, I, Act, \rightarrow, \checkmark)$  where:

- $S$  is a set of states
- $I \in S$  is the initial state
- $Act$  is a set of action labels
- $\rightarrow \subseteq S \times Act \times S$  is the transition relation
- $\checkmark \subseteq S$  is the set of **successfully terminated states**

A **deadlock state** is a state with no outgoing transitions ( $\nexists a, s'$  such that  $s \xrightarrow{a} s'$ ) AND  $s \notin \checkmark$ . Deadlock represents protocol failure wherein a packet can neither be delivered nor timed out.

For IBC, deadlock occurs when:

- A handshake remains incomplete (OpenInit sent, no OpenTry received)
- A packet is sent but neither acknowledged nor timed out

The **observable behaviour** of the system is composed of the **traces**. They are the series of actions from the starting state [28]. This formalism allows the abstraction from the inside implementation to the outside communication [60]. These traces serve as the guidelines to analyse whether a protocol follows its intended semantic interoperability.

It is important to notice when two systems with differing structure representation or graph can share the same behaviour [5]. This concept is called **behavioural equivalence** wherein the states of different LTSs are treated as equal if they exhibit the same pattern of transitions such as what the theory of bisimulation describes [5] [28]. This abstraction allows the inquiry of comparison of protocols for asset-based and data-based interoperability in blockchains without limiting particular details of their internal implementation [60].

Nevertheless, the simple usage of state transition graphs can lead to the problem of state explosion that makes formal verification difficult [67]. For this reason, there is a need to a more expressive framework that allows algebraic manipulation and composition of processes.

### 3.5 Behavioural Equivalences

In order to identify whether an implementation of a protocol matches to its expected behaviour, the concept of **behavioural equivalence** is used; particularly, the **branching bisimulation** ( $\leftrightarrow_{rb}$ ) [26] [25]. Branching simulation is needed because it maintains the branching structure of processes that is crucial in identifying deadlocks which can result to loss of value or message [26]. In order to prove this bisimulation between two different ledgers, **Cones and Foci methodology** must be used where every state in an implementation is mapped to a particular focus point at a specification that shows similar external behaviour [26].

#### 3.5.1 Trace Equivalence (Partial and Completed)

A **partial trace** is any finite sequence of observable actions from the initial state. On the otherhand, a **completed trace** is a sequence ending in a deadlock or termination state.

Two processes  $P$  and  $Q$  are **trace-equivalent** ( $P \approx_{tr} Q$ ) if  $Traces(P) = Traces(Q)$ .

**Limitation:** Trace equivalence collapses branching structure. The processes  $a.(b + c)$  (choose between b or c after a) and  $a.b + a.c$  (choose between doing a then b or a then c) have identical trace sets  $\{ab, ac\}$  but exhibit different branching behaviour at the moment of choice.

For interoperability, trace equivalence cannot distinguish a protocol that offers a choice between acknowledgement and timeout ( $ack + timeout$ ) from one that forces acknowledgement then timeout ( $ack \cdot timeout$ ).

### 3.5.2 Bisimulation (Strong and Weak)

**Strong bisimulation** ( $\leftrightarrow$ ) requires that every action of  $P$  is matched by exactly the same action of  $Q$ , including  $\tau$ . A relation  $R$  is a strong bisimulation if whenever  $(P, Q) \in R$ :

- If  $P \xrightarrow{a} P'$ , then  $\exists Q'$  such that  $Q \xrightarrow{a} Q'$  and  $(P', Q') \in R$
- If  $Q \xrightarrow{a} Q'$ , then  $\exists P'$  such that  $P \xrightarrow{a} P'$  and  $(P', Q') \in R$

**Weak bisimulation** ( $\approx$ ) abstracts over  $\tau$  steps. Define  $\Rightarrow$  as zero or more  $\tau$  steps. Then  $P \xrightarrow{a} \Rightarrow Q$  means  $P \Rightarrow \xrightarrow{a} \Rightarrow Q$ . Weak bisimulation requires matching actions after absorbing  $\tau$  steps.

For IBC, weak bisimulation is appropriate because relay forwarding ( $\tau$ ) should not affect observable correctness.

### 3.5.3 Branching Bisimulation and Cones and Foci

**Branching bisimulation** ( $\leftrightarrow_{rb}$ ) preserves the branching structure of processes, unlike trace equivalence. It is crucial for identifying deadlocks which can result in loss of value or messages [26].

To prove bisimulation between two processes (e.g., asset and data specifications), this research adopts the **Cones and Foci methodology** [26]:

- A **cone** is the set of all states reachable from a given state
- A **focus** is a state in the specification that the implementation maps to
- Construct a state mapping function  $\phi$  from implementation states to specification states
- Prove each  $\tau$ -step either preserves  $\phi$  or moves closer to a focus point

This methodology establishes branching bisimulation between IBC's concrete behaviour and its abstract specification.

## 3.6 From Process Algebra to LTS: Operational Semantics

Structural Operational Semantics (SOS) defines how process terms transition. Each operator has inference rules [68].

### 3.6.1 Atomic Action

$$\overline{a \xrightarrow{a} \checkmark}$$

### 3.6.2 Sequential Composition ( $P \cdot Q$ )

$$\frac{P \xrightarrow{a} P'}{P \cdot Q \xrightarrow{a} P' \cdot Q} \quad \frac{P \xrightarrow{a} \checkmark}{P \cdot Q \xrightarrow{a} Q}$$

### 3.6.3 Alternative Composition ( $P + Q$ )

$$\frac{P \xrightarrow{a} P'}{P + Q \xrightarrow{a} P'} \quad \frac{Q \xrightarrow{a} Q'}{P + Q \xrightarrow{a} Q'}$$

### 3.6.4 Parallel Composition ( $P \parallel Q$ )

$$\frac{P \xrightarrow{a} P'}{P \parallel Q \xrightarrow{a} P' \parallel Q} \quad \frac{Q \xrightarrow{a} Q'}{P \parallel Q \xrightarrow{a} P \parallel Q'}$$

### 3.6.5 Communication (Synchronisation)

$$\frac{P \xrightarrow{s_p(d)} P' \quad Q \xrightarrow{r_p(d)} Q'}{P \parallel Q \xrightarrow{c_p(d)} P' \parallel Q'}$$

where  $\gamma(s_p(d), r_p(d)) = c_p(d)$ .

### 3.6.6 Encapsulation ( $\partial_H(P)$ )

$$\frac{P \xrightarrow{a} P' \quad a \notin H}{\partial_H(P) \xrightarrow{a} \partial_H(P')}$$

### 3.6.7 Abstraction ( $\tau_I(P)$ )

$$\frac{P \xrightarrow{a} P' \quad a \notin I}{\tau_I(P) \xrightarrow{a} \tau_I(P')} \quad \frac{P \xrightarrow{a} P' \quad a \in I}{\tau_I(P) \xrightarrow{\tau} \tau_I(P')}$$

These rules provide the formal basis for extracting LTS models from process algebra specifications of IBC.



### 3.7 Application to Blockchain Interoperability

Process algebra has been successfully applied to blockchain systems, establishing precedent for this research.

- The  $\mathfrak{h}$ -calculus (nu-calculus) is a process calculus embedded in Haskell and formalised in Isabelle/HOL, has been used to verify the Ouroboros consensus protocol for Cardano. Key innovation: using two stacked LTSs to handle data transfer and channel scope modularly [33].
- CCS (Calculus of Communicating Systems) and weak bisimulation has been used to verify optimising transformations on smart contracts written in Cardano's Marlowe eDSL. They proved that transformed contracts preserve observable behaviour ( $P_{original} \approx P_{transformed}$ ) [54].

#### Why Process Algebra for IBC Specifically:

1. **Concurrent channels:** IBC allows multiple channels between same chains. Parallel operator ( $\parallel$ ) models this naturally.
2. **Branching choices:** IBC's handshake includes choice points ( $\text{OpenInit} \rightarrow \text{OpenTry} \text{ OR } \text{timeout}$ ). Alternative composition ( $+$ ) preserves branching structure.
3. **Silent abstraction:** Relay behaviour is internal.  $\tau$  operator and weak bisimulation allow abstraction while preserving observable correctness.
4. **Compositional analysis:** IBC separates TAO from application layer. Process algebra allows verifying assets and data separately, then composing them to detect conflicts.
5. **Behavioural equivalence:** The central question (unified/stacked/conflicting) requires weak bisimulation as the mathematical criterion.

### 3.8 Framework Synthesis

## Chapter 4

# Conceptual Framework and Methodology

This chapter bridges the theoretical framework established in Chapter 3 and the formal analysis to be presented in the next chapters. It presents the conceptual framework that operationalises the theoretical concepts for IBC specifically, followed by a step-by-step methodology for conducting the formal specification, LTS extraction, and behavioural comparison.

The chapter addresses the following questions:

- How are the theoretical concepts (atomic actions, operators, bisimulation) instantiated for IBC?
- What is the step-by-step procedure for producing the process algebra specification?
- How are LTS models extracted from process algebra terms?
- How is the unified/stacked/conflicting classification determined?

### 4.1 Conceptual Framework

The theoretical framework in Chapter 3 introduced general concepts from process algebra and LTS theory. This subsection defines those concepts specifically for the IBC protocol.

### 4.1.1 Observable Behaviour in IBC

For the purposes of this research, **observable behaviour** of IBC consists of actions that are visible to an external observer of the protocol. The following actions are considered observable:

- **Packet send events:** When a source chain commits a packet to a channel
- **Packet receive events:** When a target chain accepts a packet
- **Acknowledgement events:** When a packet is successfully processed
- **Timeout events:** When a packet fails to be delivered within the deadline
- **Handshake events:** OpenInit, OpenTry, OpenAck, OpenConfirm
- **Channel closure events:** CloseInit, CloseConfirm

The following are considered **internal (silent) actions** and are abstracted as  $\tau$ :

- Relay packet forwarding between chains
- Cryptographic proof generation and verification (Merkle proofs)
- Light client state synchronisation
- Internal state updates within a single blockchain

### 4.1.2 Trace Definition for IBC

A **trace** in IBC is a finite sequence of observable actions from the initial channel establishment to either successful termination or deadlock. For example, a successful data transfer trace is:

$$Trace_{data} = \langle OpenInit, OpenTry, OpenAck, OpenConfirm, Send, Receive, Ack \rangle$$

A trace with timeout is:

$$Trace_{timeout} = \langle OpenInit, OpenTry, OpenAck, OpenConfirm, Send, Timeout \rangle$$

### 4.1.3 Branching Structure in IBC

Branching structure occurs when the protocol has a choice between two or more continuations from a given state. In IBC, branching occurs at:

- **Acknowledgement vs Timeout:** After sending a packet, the protocol may either receive an acknowledgement (*ack*) or expire (*timeout*). This is modelled as *ack* + *timeout*.
- **Handshake acceptance vs rejection:** During channel establishment, the target chain may accept (*OpenTry*) or reject (*OpenTryReject*).

### 4.1.4 Deadlock in IBC

A **deadlock** in IBC is a state from which no further progress is possible but which is not a successful termination. Examples include:

- **Incomplete handshake:** OpenInit sent but no OpenTry received, and timeout mechanism not triggered
- **Trapped packet:** Packet sent but neither acknowledged nor timed out due to relayer failure
- **Channel identifier exhaustion:** No available channel identifiers for new connections

These deadlock states are captured in the LTS as states with no outgoing transitions that are not marked as termination states ( $\checkmark$ ).

## 4.2 Research Paradigm: Formal Modelling as Inquiry

This research adopts a **formal modelling paradigm** rather than an empirical one. The following distinctions are critical:

| Empirical Research                      | This Research (Formal Modelling)              |
|-----------------------------------------|-----------------------------------------------|
| Observes real system execution          | Constructs mathematical abstraction           |
| Measures performance or behaviour       | Derives behaviour from specification          |
| Statistical validity                    | Correctness of mapping validity               |
| Requires live network or implementation | Works from protocol standards (ICS documents) |
| Generalises from samples                | Generalises from formal proof                 |

Table 4.1: Comparison of Empirical and Formal Modelling Paradigms

Validity in this research is established through:

1. **Face validity:** The process algebra specification accurately reflects the IBC standards (ICS-004, ICS-020) as written
2. **Systematic mapping:** Every IBC action has a corresponding atomic action in the alphabet
3. **Complete SOS application:** All LTS transitions are derived using the SOS rules from Chapter 3
4. **Reproducibility:** The complete specification is provided in the appendix for independent verification

#### 4.2.1 The Three Analytical Scenarios (Unified, Stacked, Conflicting)

The central question of this research is whether IBC's asset-transfer and data-transfer behaviours are Unified, Stacked, or Conflicting. Table 4.1 provides the operational definitions for each scenario using the theoretical framework from Chapter 3.

| Scenario           | Operational Definition                                                                                                                                                 | LTS Property to Verify                                                                                                                                               |
|--------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Unified</b>     | Asset and data behaviours are weakly bisimilar: $P_{asset} \approx P_{data}$                                                                                           | LTS of asset and data have identical branching structure; every observable action in one has a matching action in the other after $\tau$ abstraction                 |
| <b>Stacked</b>     | Asset behaviour contains all data traces but adds constraints: $Traces(P_{data}) \subseteq Traces(P_{asset})$ but $P_{asset} \not\approx P_{data}$                     | All data traces appear as subsequences of asset traces; asset traces include additional lock-/mint actions; branching structures differ                              |
| <b>Conflicting</b> | Parallel composition introduces deadlocks not present in isolation: $Deadlock(P_{asset} \parallel P_{data}) \not\subseteq Deadlock(P_{asset}) \cup Deadlock(P_{data})$ | Running asset and data transfers simultaneously over the same channels creates deadlock states that do not occur when only asset or only data transfers are executed |

Table 4.2: Operational Definitions for Unified, Stacked, and Conflicting

### 4.2.2 Conceptual Framework Diagram

Figure 4.1 illustrates the overall conceptual framework, showing the flow from IBC standards to the final classification.

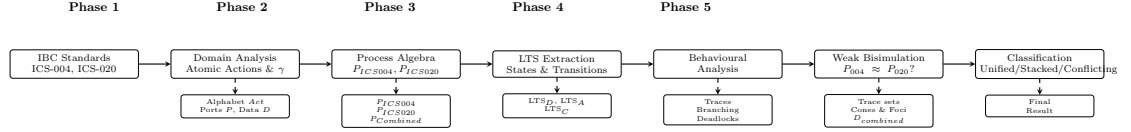


Figure 4.1: Conceptual Framework for IBC Behavioural Analysis (Five Phases)

## 4.3 Research Methodology

This research will employ a **formal specification and analytical verification** design. The methodology follows a top-down decomposition approach:

1. Start with the IBC protocol standards (ICS-004, ICS-020) as the authoritative source of truth
2. Decompose each standard into its constituent atomic actions and interaction patterns
3. Translate these patterns into process algebra terms using the operators defined in Chapter 3
4. Extract LTS models from the process algebra terms using SOS rules
5. Analyse the LTS models to identify traces, branching structures, deadlocks, and synchronisation patterns
6. Compare the asset and data behaviours using weak bisimulation and deadlock detection
7. Classify the relationship as Unified, Stacked, or Conflicting

#### Tools to be used:

- Manual process algebra specification (ACP/CCS style notation)
- Manual LTS extraction and analysis (guided by SOS rules)
- Optional: Validation using process algebra toolset (mCRL2 or CADP) if available

### 4.3.1 Phase 1: Domain Analysis and Atomic Action Identification

This phase addresses **Objective 1** and provides the foundation for **RQ1**.

#### 1. Decompose IBC Standards

Obtain the authoritative IBC standards:

- ICS-004: Channel and Packet Semantics (data transport layer)
- ICS-020: Fungible Token Transfer (asset transfer layer)

Sources: Cosmos IBC documentation and specification repositories [19] [18].

#### 2. Identify Communication Ports

Define the set of communication ports  $P$  for IBC:

- $p_{chainA}$ : Port on source blockchain
- $p_{chainB}$ : Port on target blockchain
- $p_{relayer}$ : Port for off-chain relayer (internal, will be abstracted)
- $p_{channel}$ : Logical channel endpoint

#### 3. Define Data Types

Define the set of data  $D$  for IBC packets:

- $d_{packet}$ : General packet payload (bytes, payload-agnostic)
- $d_{asset}$ : Asset transfer payload (amount, denomination, sender, receiver)
- $d_{ack}$ : Acknowledgement data (success/failure, optional return data)
- $d_{channel\_id}$ : Channel identifier
- $d_{sequence}$ : Packet sequence number
- $d_{timeout}$ : Timeout height and timestamp

#### 4. Define Atomic Actions

Define the atomic actions for IBC:

**Handshake actions (ICS-004):** $chan\_open\_init_A(id)$  $chan\_open\_try_B(id)$  $chan\_open\_ack_A(id)$  $chan\_open\_confirm_B(id)$ **Packet actions (ICS-004):** $send\_packet_A(p, d)$  $recv\_packet_B(p, d)$  $ack_B(p, d_{ack})$  $recv\_ack_A(p, d_{ack})$  $timeout_A(p)$ **Asset-specific actions (ICS-020):** $lock_A(asset)$  $burn_A(asset)$  (alternative to lock) $mint_B(voucher)$  $verify\_proof_B(proof)$ **5. Define Communication Function**

Define the communication function  $\gamma$  that pairs send and read actions:

$$\gamma(send\_packet_A(p, d), recv\_packet_B(p, d)) = c\_packet(p, d)$$

$$\gamma(ack_B(p, d_{ack}), recv\_ack_A(p, d_{ack})) = c\_ack(p, d_{ack})$$

$$\gamma(chan\_open\_init_A(id), chan\_open\_try_B(id)) = c\_open(id)$$

$$\gamma(chan\_open\_ack_A(id), chan\_open\_confirm_B(id)) = c\_confirm(id)$$

Mismatched or isolated actions (send without matching read, or vice versa) are blocked by the encapsulation operator  $\partial_H$ .

**6. Deliverable**

A complete alphabet specification containing:



- Set of ports  $P$
- Set of data types  $D$
- Set of atomic actions  $Act = Act_{handshake} \cup Act_{packet} \cup Act_{asset}$
- Communication function  $\gamma$
- Set of internal actions  $I$  to be abstracted (relay forwarding, proof verification)

### 4.3.2 Phase 2: Process Algebra Specification

This phase addresses **Objective 2** and completes **RQ1**.

#### 1. Specify ICS-004 (Data Transport)

Define the channel handshake process:

$$Handshake = chan\_open\_init_A \cdot (chan\_open\_try_B \cdot chan\_open\_ack_A \cdot chan\_open\_confirm_B + TimeoutReject)$$

Define the packet flow process:

$$PacketFlow = send\_packet_A \cdot (recv\_packet_B \cdot ack_B \cdot recv\_ack_A + timeout_A)$$

Define the full channel process (recursive for multiple packets):

$$Channel = Handshake \cdot (PacketFlow)^* \cdot Close$$

where  $(PacketFlow)^*$  denotes zero or more repetitions of  $PacketFlow$ , and  $Close$  terminates the channel.

Define the complete IBC data transport system:

$$IBC_{Data} = \partial_H(Chain_A \parallel Relayer \parallel Chain_B)$$

where:

- $Chain_A$  and  $Chain_B$  are blockchain processes running  $Channel$
- $Relayer$  forwards packets internally (modelled as  $\tau$  actions)
- $\partial_H$  blocks unmatched send/read actions (requires handshake completion)

## 2. Specify ICS-020 (Asset Transfer)

Define the lock/burn process on source chain:

$$LockProcess = lock_A(asset) \cdot verify\_lock_A \cdot packet\_prep$$

Define the mint/claim process on target chain:

$$MintProcess = verify\_proof_B \cdot mint_B(voucher) \cdot ack\_send$$

Define the full asset transfer (wrapped in ICS-004 packet):

$$AssetTransfer = LockProcess \cdot SendPacket \cdot MintProcess + Rollback$$

where *Rollback* handles timeout or verification failure by unlocking the asset.

Define the complete IBC asset transfer system:

$$IBC_{Asset} = \partial_H(ChainA_{asset} \parallel Relayer \parallel ChainB_{asset})$$

## 3. Specify Combined System (for Conflict Detection)

To test for conflicts, define the parallel composition of asset and data flows over the same channels:

$$IBC_{Combined} = \partial_H((Chain_A \parallel ChainA_{asset}) \parallel Relayer \parallel (Chain_B \parallel ChainB_{asset}))$$

## 4. Deliverable

Complete process algebra specifications for:

- $P_{ICS004}$  (data transport only)
- $P_{ICS020}$  (asset transfer only)
- $P_{Combined}$  (both running concurrently)

All specifications follow the syntax defined in Chapter 3 and will be included in the appendices.

### 4.3.3 Phase 3: LTS Extraction

This phase addresses **Objective 3** and **RQ2**.

#### 1. Generate LTS from Process Terms

Using the SOS rules defined in Chapter 3 (Section 3.6), generate the LTS for each process term:

**Procedure:**

- (1) Start from the initial state of the process term
- (2) For each state, examine all possible SOS rule applications
- (3) For each applicable rule, derive the next state and transition label
- (4) Add the new state to the set  $S$  and the transition to  $\rightarrow$
- (5) Repeat until no new states are reachable (fixpoint)
- (6) Mark termination states as  $\checkmark$
- (7) Mark deadlock states (no outgoing transitions, not terminated) as  $\delta$

#### 2. Record LTS Properties

For each LTS ( $LTS_{Data}$ ,  $LTS_{Asset}$ ,  $LTS_{Combined}$ ), record:

- (1) **States (S):** List of all reachable states with unique identifiers
- (2) **Transitions ( $\rightarrow$ ):** Labelled edges between states
- (3) **Partial traces:** All finite sequences of observable actions
- (4) **Completed traces:** Sequences ending in  $\checkmark$  or  $\delta$
- (5) **Branching points:** States with multiple outgoing transitions (choice points)
- (6) **Deadlock states:** States with no outgoing transitions and not marked  $\checkmark$
- (7) **Synchronisation patterns:** Transitions labelled with communication actions  $c_p(d)$

#### 3. Visualise LTS

Produce transition graphs for each LTS. Example structure for  $LTS_{Data}$ :

```

State S0 (Initial) --chan_open_init_A--> S1
State S1 --chan_open_try_B--> S2
State S2 --chan_open_ack_A--> S3
State S3 --chan_open_confirm_B--> S4 (Channel Open)
State S4 --send_packet_A--> S5
State S5 --recv_packet_B--> S6
State S6 (Choice) --ack_B--> S7 or --timeout_A--> S8
...

```

#### 4. Deliverable

For each specification ( $P_{ICS004}$ ,  $P_{ICS020}$ ,  $P_{Combined}$ ):

- Complete LTS with enumerated states and transitions
- Set of partial traces
- Set of completed traces
- Set of deadlock states (if any)
- Branching structure documentation

#### 4.3.4 Phase 4: Behavioural Comparison

This phase addresses **Objectives 4 and 5** and answers **RQ3**.

##### 1. Compare Trace Sets

Compute and compare trace sets:

$$T_{data} = Traces(P_{ICS004})$$

$$T_{asset} = Traces(P_{ICS020})$$

Check:

- (1)  $T_{data} \subseteq T_{asset}$ ? (Are all data traces also asset traces?)
- (2)  $T_{asset} \subseteq T_{data}$ ? (Are all asset traces also data traces?)

(3)  $T_{data} = T_{asset}$ ? (Are the trace sets identical?)

**Interpretation:**

- If  $T_{data} = T_{asset} \rightarrow$  possible Unified (but must check branching)
- If  $T_{data} \subset T_{asset} \rightarrow$  possible Stacked
- If neither inclusion holds  $\rightarrow$  likely Conflicting or behaviours fundamentally different

2. Check Weak Bisimulation

Apply the Cones and Foci methodology [26] to determine if  $P_{ICS004} \approx P_{ICS020}$ :

**Procedure:**

- (1) Construct a state mapping function  $\phi$  from states of  $P_{ICS020}$  to states of  $P_{ICS004}$
- (2) For each state in  $P_{ICS020}$ , identify the matching "focus" state in  $P_{ICS004}$
- (3) Prove that for each visible action, the branching structure is preserved
- (4) Prove that  $\tau$  steps in one process can be matched by zero or more  $\tau$  steps in the other

If  $\phi$  exists and all proof obligations are satisfied, then  $P_{ICS004} \approx P_{ICS020}$  (weak bisimilar).

3. Detect Conflicts

Compute deadlock sets:

$$D_{data} = \text{Deadlock}(P_{ICS004})$$

$$D_{asset} = \text{Deadlock}(P_{ICS020})$$

$$D_{combined} = \text{Deadlock}(P_{ICS004} \parallel P_{ICS020})$$

Check if:

$$D_{combined} \not\subseteq D_{data} \cup D_{asset}$$

If YES, then new deadlocks appear only when asset and data behaviours run concurrently.

For each new deadlock, document the trace that leads to it.

4. Assign Final Classification

Apply the decision rules from Table 4.1:

## 5. Deliverable

- Trace inclusion relationships (with evidence)
- Weak bisimulation proof or disproof (with Cones and Foci mapping)
- Deadlock sets for all three processes
- Final classification: **Unified**, **Stacked**, or **Conflicting**
- If Conflicting: Documented traces leading to new deadlocks

## 4.3.5 Phase 5: Documentation and Reusability

This phase addresses **Objective 6**.

## 1. Standardise Notation

Legends for all process algebra symbols used:

| Symbol                 | Meaning                                          |
|------------------------|--------------------------------------------------|
| $a$                    | Atomic action                                    |
| $\cdot$                | Sequential composition                           |
| $+$                    | Alternative composition (choice)                 |
| $\parallel$            | Parallel composition                             |
| $\partial_H$           | Encapsulation (block actions in set $H$ )        |
| $\tau_I$               | Abstraction (rename actions in $I$ to $\tau$ )   |
| $\tau$                 | Silent (internal) action                         |
| $\delta$               | Deadlock                                         |
| $\checkmark$           | Successful termination                           |
| $\gamma$               | Communication function                           |
| $s_p(d)$               | Send action on port $p$ with data $d$            |
| $r_p(d)$               | Read action on port $p$ with data $d$            |
| $c_p(d)$               | Communication action (synchronised send/receive) |
| $\approx$              | Weak bisimulation                                |
| $\leftrightarrow_{rb}$ | Branching bisimulation                           |

Table 4.3: Process Algebra Notation Legend

## 2. Document Assumptions and Abstractions

List of all abstractions made (from Scope and Limitations in Chapter 1):

- (1) Cryptographic verification (Merkle proofs, light client sync) is abstracted as  $\tau$
- (2) Economic incentives (gas costs, slashing, liquidity constraints) are not modelled

- (3) Relay behaviour is abstracted as  $\tau$  (only observable effects are packet delivery)
- (4) Only ICS-004 and ICS-020 are modelled; ICS-27 (Interchain Accounts) and ICS-31 (Interchain Queries) are excluded
- (5) Security properties beyond observable behaviour (e.g., double-spending resistance) are not verified
- (6) No empirical validation or live protocol execution is performed

### 3. Produce Reusable Specification

Format the complete specification as an appendix with:

- Complete alphabet definition
- Complete process algebra terms for  $P_{ICS004}$ ,  $P_{ICS020}$ , and  $P_{Combined}$
- Communication function  $\gamma$  definition
- Encapsulation set  $H$  and abstraction set  $I$
- Commentary explaining each part of the specification

This specification is designed to be reusable for:

- Future comparison with Polkadot XCM
- Future comparison with Axelar General Message Passing (GMP)
- Input to automated process algebra tools (mCRL2, CADP, or similar)

### 4. Deliverable

- An appendix for complete and commented process algebra specification
- An appendix for the extracted LTS models (states and transitions)
- Documentation of assumptions for future researchers

## 4.4 Validation and Quality Criteria

Unlike empirical research, formal modelling research requires different validation criteria.

#### 4.4.1 Correctness of Formalisation

- **Face validity:** The process algebra specification is reviewed against ICS-004 and ICS-020 documents line by line. Every protocol action in the standards has a corresponding atomic action or process term in the specification.
- **Systematic mapping:** A mapping table documents how each IBC standard element translates to process algebra.
- **Expert review (if available):** A researcher with formal methods or blockchain interoperability expertise reviews the specification for accuracy.

#### 4.4.2 Completeness of LTS Extraction

- **State space coverage:** The LTS includes all states reachable from the initial state via SOS rule application. No reachable state is omitted.
- **Termination handling:** All termination states ( $\checkmark$ ) are correctly identified.
- **Deadlock detection:** All deadlock states are identified and documented.

#### 4.4.3 Reproducibility

- The complete process algebra specification is provided in the appendices.
- Each step of LTS generation is documented (transition-by-transition)
- If tools are used (mCRL2, CADP), the input scripts and commands are included
- The methodology is described with sufficient detail for another researcher to replicate

#### 4.4.4 Limitations

- **Abstraction may hide subtle bugs:** Cryptographic failures (e.g., Merkle proof forgery) are not modelled; economic attacks (e.g., relayer bribery) are not captured; timing-dependent issues may be lost in  $\tau$  abstraction
- **Manual LTS extraction:** Manual derivation may introduce human error (mitigated by systematic SOS application and cross-checking)



- **Scalability:** State explosion may occur for highly concurrent scenarios (mitigated by compositional analysis and abstraction)
- **No empirical validation:** Results are theoretical predictions, not guarantees about live IBC implementations

## 4.5 Ethical Considerations

This research involves no human subjects, no personal data collection, and no live blockchain interaction. The formal modelling and analysis are conducted entirely on publicly available protocol specifications (ICS-004, ICS-020). No economic or security risks are introduced as no actual assets or live transactions are involved. Proper attribution is given to IBC standard authors, process algebra pioneers, and all cited researchers.

## 4.6 Summary

This chapter has presented the conceptual framework and methodology for analysing IBC's asset and data behaviours. The key elements are:

- **Conceptual framework:** Operational definitions of observable behaviour, traces, branching structure, and deadlock specifically for IBC; three analytical scenarios (Unified, Stacked, Conflicting) with formal criteria; conceptual diagram showing the analysis flow
- **Methodology:** Five phases: (1) domain analysis and atomic action identification, (2) process algebra specification, (3) LTS extraction, (4) behavioural comparison using weak bisimulation and deadlock detection, (5) documentation for reusability
- **Validation:** Correctness through face validity and systematic mapping; completeness through state space coverage; reproducibility through complete specification in appendix

## Appendix A

# Use of Generative AI

The proponents utilised three generative AI: ChatGPT, Deepseek, NotbeookLM. However, there are no set rules on which to use for specific topics. The proponents use either of the three depending on their convenience.

### A.1 Brainstorming

Since generative AI tools like ChatGPT can quickly provide answers, it was used to ask trends that are happening in the field of Blockchain. It was asked to provide recent published papers that were checked by the proponents. It would give several research titles with the authors and links. The proponents would look at each of them to ensure their legitimacy. Extraction of the content of the provided researches were not used as there was no guarantee that the AI would give correct results. Tasks like those are prone to hallucinations.

### A.2 Searching

Another usage was the creation of keywords that can be used to search for papers from the online databases found in the internet. This made the curation of papers easier and faster. This, however, did not replace the manual reading of the existing papers especially their RRL section to find keywords, know more about the topic, and look for possible mentioned references.

### A.3 Formatting

During the writing of this paper, AI was used to automatically convert comma separated values into tables using latex syntax. This was done for convenience since the actual data was already created by the proponents.

Whenever a reference does not contain a DOI but contains its APA citation, ChatGPT is used to translate the APA format into bibtex format. This also applies to other citation formats to bibtex.

### A.4 Debugging

AI was also used in debugging error messages given by latex compiler during the compilation of the PDF format.

# Bibliography

- [1] A. A. ALZHRANI, A. Y. NOAMAN, A. A. A. GAD-ELRAB, F. E. EASSA, M. KHEMAKHEM, F. ALBALWY, AND H. ALJIHANI, *A comprehensive review of interoperability challenges and applications beyond cryptocurrencies*, Advances in Artificial Intelligence and Machine Learning, 05 (2025), p. 3373–3388.
- [2] M. T. ALAM, R. HALDER, AND A. MAITI, *Formal verification of the pub-sub blockchain interoperability protocol using stochastic timed automata*, Frontiers in Blockchain, 6 (2023).
- [3] A. AUGUSTO, R. BELCHIOR, A. VASCONCELOS, T. HARDJONO, AND M. CORREIA, *Multi-party cross-chain asset transfers*, (2023).
- [4] A. AUGUSTO, A. VASCONCELOS, M. CORREIA, AND L. ZHANG, *Xchaindatagen: A cross-chain dataset generation framework*, 2025.
- [5] J. C. M. BAETEN AND C. VERHOEF, *Concrete Process Algebra*, no. 9503 in Computing Science Reports, Technische Universiteit Eindhoven, Eindhoven, Netherlands, 1995.
- [6] R. BELCHIOR, L. RILEY, T. HARDJONO, A. VASCONCELOS, AND M. CORREIA, *Do you need a distributed ledger technology interoperability solution?*, Distributed Ledger Technologies: Research and Practice, 2 (2023), p. 1–37.
- [7] R. BELCHIOR, A. VASCONCELOS, S. GUERREIRO, AND M. CORREIA, *A survey on blockchain interoperability: Past, present, and future trends*, ACM Computing Surveys, 54 (2021), p. 1–41.
- [8] R. BELCHIOR, A. VASCONCELOS, S. GUERREIRO, AND M. CORREIA, *A survey on blockchain interoperability: Past, present, and future trends*, ACM Computing Surveys (CSUR), 54 (2021), pp. 1–41.
- [9] N. BELENKOV, V. CALLENS, A. MURASHKIN, K. BAK, M. DERKA, J. GORZNY, AND S.-S. LEE, *Sok: A review of cross-chain bridge hacks in 2023*, (2025).
- [10] J. A. BERGSTRA AND J. W. KLOP, *Fixed point semantics in process algebras*, 1982.
- [11] A. BIGIOTTI, L. MOSTARDA, A. NAVARRA, A. PINNA, D. SESTILI, R. TONELLI, AND M. VACCARGIU, *Blockchain agnostic protocols: An analysis of the state of the art*, ACM Computing Surveys, 58 (2025), p. 1–36.
- [12] C. BOYD, K. GJØSTEEN, AND S. WU, *A blockchain model in tamarin and formal analysis of hash time lock contract*, vol. 84, Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 2020, pp. 5:1–5:13.

- [13] S. BRAITHWAITE, E. BUCHMAN, I. KONNOV, Z. MILOSEVIC, I. STOILKOVSKA, J. WIDDER, AND A. ZAMFIR, *Formal specification and model checking of the tendermint blockchain synchronization protocol (short paper)*, vol. 84, Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 2020, pp. 10:1–10:8.
- [14] P. CAETANO DA SILVA, G. A. L. D. CASTRO, AND D. J. DIAZ, *Interoperability between blockchain platforms with cross-chain*, Contabilidad y Negocios, 20 (2025), p. 145–185.
- [15] L. CHEN, Z. YAO, X. SI, AND Q. ZHANG, *Three-stage cross-chain protocol based on notary group*, Electronics, 12 (2023), p. 2804.
- [16] L. CHENG, Z. LV, O. ALFARRAJ, A. TOLBA, X. YU, AND Y. REN, *Secure cross-chain interaction solution in multi-blockchain environment*, Heliyon, 10 (2024), p. e28861.
- [17] J. O. CHERVINSKI, D. KREUTZ, X. XU, AND J. YU, *Analyzing the performance of the inter-blockchain communication protocol*, in 2023 53rd Annual IEEE/IFIP International Conference on Dependable Systems and Networks (DSN), IEEE, June 2023, p. 151–164.
- [18] COSMOS CONTRIBUTORS, *Ibc: Interchain standards for the cosmos network and interchain ecosystem*, 2026. Accessed: 2026-05-11.
- [19] COSMOS DOCUMENTATION, *Introduction to ibc*. <https://docs.cosmos.network/ibc/latest/intro>, 2024. Accessed: 2026-05-07.
- [20] L. P. A. DA HORTA, J. S. REIS, M. PEREIRA, AND S. M. DE SOUSA, *Why3son: Proving your michelson smart contracts in why3*, 2020.
- [21] J. V. B. DA SILVA, A. GUPTA, AND M. S. I. SINGH, *Qlink: Quantum-safe bridge architecture for blockchain interoperability*, 2025.
- [22] C. DELGADO-VON-EITZEN, L. ANIDO-RIFÓN, M. RUIZ-MOLINA, AND M. J. FERNÁNDEZ-IGLESIAS, *Bridging the gap: Achieving seamless interoperability between ethereum-based blockchains using inter-blockchain communication protocols*, Software: Practice and Experience, 55 (2025), p. 1749–1772.
- [23] Z. DENG, C. TANG, T. LI, P. ABLA, Q. CHEN, W. LIANG, AND D. HE, *Enhancing blockchain cross chain interoperability: A comprehensive survey*, 2025.
- [24] Z. DUAN, H. MAO, Z. CHEN, X. BAI, K. HU, AND J.-P. TALPIN, *Formal modeling and verification of blockchain system*, in Proceedings of the 10th International Conference on Computer Modeling and Simulation, ICCMS 2018, ACM, Jan. 2018, p. 231–235.
- [25] W. FOKKINK, *Introduction to Process Algebra*, Computer Science – Monograph, Springer-Verlag, Berlin, Heidelberg, New York, London, Paris, Tokyo, Hong Kong, Barcelona, Budapest, 2nd ed., 2007. English edition, published April 10, 2007.
- [26] W. J. FOKKINK, J. F. GROOTE, AND M. A. RENIERS, *Process algebra needs proof methodology*, 2004.
- [27] C. GOES, *The interblockchain communication protocol: An overview*, arXiv preprint arXiv:2006.15918, (2020).

- [28] R. GORRIERI, *Process Algebras for Petri Nets*, Springer International Publishing, 2017, p. 1–14.
- [29] C. G. HARRIS, *Cross-chain technologies: Challenges and opportunities for blockchain interoperability*, in 2023 IEEE International Conference on Omni-layer Intelligent Systems (COINS), IEEE, July 2023, p. 1–6.
- [30] K. HASSAN, A. NIK, A. SOKHANKHOSH, K. ESMAEILZADEH KHORASANI, S. MORADI, AND S. ROUHANI, *Sok: Systematizing blockchain interoperability through layered architecture and layer 2 interoperability*, Distributed Ledger Technologies: Research and Practice, (2025).
- [31] IBC PROTOCOL DOCUMENTATION, *What are the ibc transport and application layers?* <https://ibcprotocol.dev/blog/what-are-the-ibc-transport-and-application-layers>, 2024. Accessed: 2026-05-07.
- [32] ———, *How the ibc protocol works*. <https://ibcprotocol.dev/how-ibc-works>, 2025. Accessed: 2026-05-07.
- [33] W. JELTSCH, *A process calculus for formally verifying blockchain consensus protocols*, 2019.
- [34] F. KAMMÜLLER AND U. NESTMANN, *Inter-blockchain protocols with the isabelle infrastructure framework*, vol. 84, Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 2020, pp. 11:1–11:12.
- [35] I. KANG, A. GUPTA, AND O. SENEVIRATNE, *Blockchain interoperability landscape*, 2022.
- [36] J. KIM, *Safety and liveness of cross-domain state preservation under byzantine faults: A mechanized proof in isabelle/hol*, 2026.
- [37] R. S. KISHORE, M. SHARMA, A. VERMA, AND R. DUBEY, *Interoperability & cross-chain frameworks*, Journal of Blockchain Systems and Smart Contracts, 1 (2025). Rohan S Kishore: Assistant Professor, Mitali Sharma: Associate Professor, Anuj Verma: Associate Professor, Rupesh Dubey: Associate Professor, Department of Crypto Systems and Security, Madurai Kamaraj University – College of Arts & Science.
- [38] S. D. KOTEY, E. T. TCHAO, A. S. AGBEMENU, A.-R. AHMED, AND E. KEELSON, *A framework for full decentralization in blockchain interoperability*, Sensors, 24 (2024), p. 7630.
- [39] S. D. KOTEY, E. T. TCHAO, A. AHMED, A. S. AGBEMENU, H. NUNOO-MENSAH, A. SIKORA, D. WELTE, AND E. KEELSON, *Blockchain interoperability: the state of heterogenous blockchain-to-blockchain communication*, IET Communications, 17 (2023), p. 891–914.
- [40] V. KUMAR, I. BUDHIRAJA, A. JABBARI, D. GARG, D. SINGH, AND N. MENGANI, *Efficient blockchain interoperability design for cross-chain transactions in future internet-of-value*, Peer-to-Peer Networking and Applications, 18 (2025).
- [41] G. LALLI, *Defining interoperability: a universal standard*, 2024.
- [42] K. LI, C. WANG, X. FENG, AND S. WU, *Blockchain interoperability in data exchange logistics integration*, Mathematics, 12 (2024), p. 1516.
- [43] L. LI, J. WU, AND W. CUI, *A review of blockchain cross-chain technology*, IET Blockchain, 3 (2023), p. 149–158.

- [44] W. LI, Z. LIU, J. CHEN, Z. LIU, AND Q. HE, *Towards blockchain interoperability: a comprehensive survey on cross-chain solutions*, Blockchain: Research and Applications, 6 (2025), p. 100286.
- [45] G. LLAMBÍAS, L. GONZÁLEZ, AND R. RUGGIA, *Blockchain interoperability patterns*, in 2024 IEEE 21st International Conference on Software Architecture Companion (ICSA-C), IEEE, June 2024, p. 127–134.
- [46] ———, *Formalising a gateway-based blockchain interoperability solution with event-b*, in 2024 IEEE International Conference on Blockchain and Cryptocurrency (ICBC), IEEE, May 2024, p. 1–9.
- [47] ———, *An approach to formalise blockchain interoperability patterns*, in 2025 IEEE International Conference on Blockchain and Cryptocurrency (ICBC), IEEE, June 2025, p. 1–9.
- [48] A. LOCHBIHLER AND O. MARIĆ, *Authenticated data structures as functors in isabelle/hol*, vol. 84, Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 2020, pp. 6:1–6:15.
- [49] A. LOHACHAB, S. GARG, B. KANG, M. B. AMIN, J. LEE, S. CHEN, AND X. XU, *Towards interconnected blockchains: A comprehensive review of the role of interoperability among disparate blockchains*, ACM Computing Surveys, 54 (2021), p. 1–39.
- [50] H. LU, A. JAJOO, AND K. S. NAMJOSHI, *Atomicity and abstraction for cross-blockchain interactions*, 2024.
- [51] M. MADINE, K. SALAH, R. JAYARAMAN, Y. AL-HAMMADI, J. ARSHAD, AND I. YAQOOB, *appxchain: Application-level interoperability for blockchain networks*, IEEE Access, 9 (2021), pp. 87777–87791. Received June 1, 2021; accepted June 10, 2021; published June 15, 2021; current version June 24, 2021.
- [52] H. MAO, T. NIE, H. SUN, D. SHEN, AND G. YU, *A survey on cross-chain technology: Challenges, development, and prospect*, IEEE Access, 11 (2023), p. 45527–45546.
- [53] F. MARIĆ, B. SCHOLZ, AND P. SUBOTIĆ, *Formal verification of a fail-safe cross-chain bridge*, vol. 129, Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 2025, pp. 8:1–8:18.
- [54] K. MCILWAINE AND J. CALDWELL, *Verifying smart contract transformations using bisimulations*, vol. 129, Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 2025, pp. 9:1–9:19.
- [55] R. MILNER, *Communication and Concurrency*, Prentice-Hall International Series in Computer Science, Prentice Hall, illustrated ed., 1989.
- [56] S. S. MOHAMMED ABDUL, A. SHRESTHA, AND J. YONG, *Crossdefi: A novel cross-chain communication protocol*, Future Internet, 16 (2024), p. 314.
- [57] J. S. NOTLAND, J. LI, M. NOWOSTAWSKI, AND P. H. HARO, *Sok: cross-chain bridging architectural design flaws and mitigations*, Blockchain: Research and Applications, 7 (2026), p. 100315.
- [58] W. OU, S. HUANG, J. ZHENG, Q. ZHANG, G. ZENG, AND W. HAN, *An overview on cross-chain: Mechanism, platforms, challenges and advances*, Computer Networks, 218 (2022), p. 109378.

- [59] B. PILLAI, K. BISWAS, Z. HOU, AND V. MUTHUKKUMARASAMY, *Cross-blockchain technology: Integration framework and security assumptions*, IEEE Access, 10 (2022), p. 41239–41259.
- [60] G. D. PLOTKIN, *A structural approach to operational semantics*. Preprint submitted to Journal of Logic and Algebraic Programming, 2004. Submitted on 30 January 2004.
- [61] QUASAR FINANCE, *Quasar: A protocol for interchain asset management*. <https://docs-staging.quasar.fi/QuasarLitepaper2023.pdf>, March 2023. Litepaper; Accessed: 2026-05-07.
- [62] K. RUPIC, L. ROŽIĆ, AND A. DEREK, *Mechanized formal model of bitcoin’s blockchain validation procedures*, vol. 84, Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 2020, pp. 7:1–7:14.
- [63] M. SALEH, S. AMANZHOLOVA, AND A. ALMISREB, *Blockchain interoperability: challenges and solutions for a connected future*, in Proceedings of the 9th International Conference on Digital Technologies in Education, Science and Industry (DTESI 2024), Almaty, Kazakhstan, october 2024. Corresponding author: mohamedswm@yahoo.com (M. Saleh); equal contributions by S. Amanzholova and A. Almisreb. ORCID: 0000-0002-6779-9393 (S. Amanzholova), 0000-0001-7581-5747 (A. Almisreb). Licensed under CC BY 4.0.
- [64] M. SOBER, M. SIGWART, P. FRAUENTHALER, C. SPANRING, M. KOBELT, AND S. SCHULTE, *Decentralized cross-blockchain asset transfers with transfer confirmation*, Cluster Computing, 26 (2022), p. 2129–2146.
- [65] S. SOTTO-MAYOR, R. BELCHIOR, M. CORREIA, AND A. VASCONCELOS, *An enterprise architecture approach to semantic blockchain interoperability*, in Proceedings of the 26th International Conference on Enterprise Information Systems (ICEIS 2024), vol. 2, 2024, pp. 614–621.
- [66] F. H. SOURESHJANI AND J. GORZNY, *Comparative analysis of cross-chain token standards*, 2026.
- [67] F. W. VAANDRAGER, *Verification of two communication protocols by means of process algebra*, 1986.
- [68] F. W. VAANDRAGER, *Two simple protocols*, Cambridge University Press, Sept. 1990, p. 23–44.
- [69] J. F. A. K. VAN BENTHEM AND J. A. BERGSTRA, *Logic of transition systems*, Journal of Logic, Language and Information, 3 (1995), pp. 247–283.
- [70] S. VERMA, D. YADAV, AND G. CHANDRA, *Introduction of formal methods in blockchain consensus mechanism and its associated protocols*, IEEE Access, 10 (2022), pp. 67858–67873. Received April 11, 2022; accepted June 4, 2022; published June 21, 2022; current version June 27, 2022.
- [71] A. VESCHETTI, R. BUBEL, AND R. HÄHNLE, *Smartml: Towards a modeling language for smart contracts*, 2024.
- [72] F. WANG, *An approach to formal verification of atomic swap protocols*, in 2nd Workshop on Intelligent Infocommunication Networks, Systems and Services, Budapest University of Technology and Economics, 2024, p. 55–60.



- [73] G. WANG, *SoK: Exploring blockchains interoperability*. Cryptology ePrint Archive, Paper 2021/537, 2021.
- [74] G. WANG, Q. WANG, AND S. CHEN, *Exploring blockchains interoperability: A systematic survey*, ACM Computing Surveys, 55 (2023), p. 1–38.
- [75] ———, *Exploring blockchains interoperability: A systematic survey*, ACM Computing Surveys, 55 (2023), p. 1–38.
- [76] W. WANG, Z. ZHANG, G. WANG, AND Y. YUAN, *Efficient cross-chain transaction processing on blockchains*, Applied Sciences, 12 (2022), p. 4434.
- [77] Q. WEI, X. ZHAO, X.-Y. ZHU, AND W. ZHANG, *Formal analysis of ibc protocol*, in 2023 IEEE 31st International Conference on Network Protocols (ICNP), IEEE, Oct. 2023, p. 1–11.
- [78] S. WILSON, K. ADU-DUODU, Y. LI, E. SOLAIMAN, O. RANA, AND R. RANJAN, *Exploring blockchain interoperability: Frameworks, use cases, and future challenges*, 2026.
- [79] G. WOOD, *Ethereum: A secure decentralised generalised transaction ledger*, tech. rep., Ethereum Foundation, 2021.
- [80] X. WU, T. ZHANG, J. WANG, J. CHENG, AND Z. WANG, *A distributed cross-chain mechanism based on notary schemes and group signatures*, Journal of King Saud University - Computer and Information Sciences, 35 (2023), p. 101862.